

DG - repetition - no_frac - Serial position analysis

```
# do this in calling R script
# library(ggplot2)
# library(fmsb) # for TestModels
# library(lme4)
# library(kableExtra)
# library(MASS) # for dropterm
# library(tidyverse)
# library(dominanceanalysis)
# library(cowplot) # for multiple plots in one figure
# library(formatters) # to wrap strings for plot titles
# library(pals) # for continuous color palettes

# load needed functions
# source(paste0(RootDir, "/src/function_library/sp_functions.R"))
# do this in the calling R script

# for testing
# CurPat <- "GM"
# CurTask <- "repetition"
# MinLength <- 4
# MaxLength <- 10
# BestModelIndexL1 <- 1
# BestModelIndexL2 <- 1
# BestModelIndexL3 <- 1
# ReportTitle <- paste(CurPat, " - ", CurTask, " - ", RunLabel, " - Serial position analysis")
# RandomSamples <- 10
# DoSimulations <- FALSE
# RemoveFracPres <- TRUE

CurPat <- params$CurPat
CurTask <- params$CurTask
MinLength <- params$MinLength
MaxLength <- params$MaxLength
BestModelIndexL1 <- params$BestModelIndexL1
BestModelIndexL2 <- params$BestModelIndexL2
BestModelIndexL3 <- params$BestModelIndexL3
ReportTitle <- params$ReportTitle
RandomSamples <- params$RandomSamples
DoSimulations <- params$DoSimulations
RemoveFracPres <- params$RemoveFracPres

# done in calling script
# print(paste0("Using root dir: ", RootDir))
# RunDir <- paste0(paste0(RootDir, "/output/"), RunLabel))
# if(!dir.exists(RunDir)){
#   dir.create(RunDir, showWarnings = TRUE)
#   print(paste0("created run directory: ", RunDir))
# }
```

```

# # create patient directory if it doesn't already exist
# PatientDir <- paste0(RootDir, "/output/", RunLabel, "/", CurPat)
# if(!dir.exists(PatientDir)){
#   dir.create(PatientDir, showWarnings = TRUE)
#   print(paste0("created participant directory: ", PatientDir))
# }
# TablesDir <- paste0(RootDir, "/output/", RunLabel, "/", CurPat, "/tables/")
# if(!dir.exists(TablesDir)){
#   dir.create(TablesDir, showWarnings = TRUE)
#   print(paste0("created tables directory: ", TablesDir))
# }
# FigDir <- paste0(RootDir, "/output/", RunLabel, "/", CurPat, "/fig/")
# if(!dir.exists(FigDir)){
#   dir.create(FigDir, showWarnings = TRUE)
#   print(paste0("created figures directory: ", FigDir))
# }
# ReportsDir <- paste0(RootDir, "/output/", RunLabel, "/", CurPat, "/reports/")
# if(!dir.exists(ReportsDir)){
#   dir.create(ReportsDir, showWarnings = TRUE)
#   print(paste0("created reports directory: ", ReportsDir))
# }
results_report_DF <- data.frame(Description=character(), ParameterValue=double())

ModelDatFilename<-paste0(RootDir, "/data/", CurPat, "/", CurPat, "_", CurTask, "_posdat.csv")
if(!file.exists(ModelDatFilename)){
  print("**ERROR** Input data file not found")
  print(sprintf("\tfile: ", ModelDatFilename))
  print(sprintf("\troot dir: ", RootDir))
}
PosDat<-read.csv(ModelDatFilename)

# may already be done in datafile
# if(RemoveFinalPosition){
#   PosDat <- PosDat[PosDat$pos < PosDat$stimlen,] # remove final position
# }
PosDat <- PosDat[PosDat$stimlen >= MinLength,] # include only lengths with sufficient N
PosDat <- PosDat[PosDat$stimlen <= MaxLength,]
# include only correct and nonword errors (not no response, word or w+nw errors)
PosDat <- PosDat[(PosDat$err_type == "correct") | (PosDat$err_type == "nonword"), ]
PosDat <- PosDat[(PosDat$pos) != (PosDat$stimlen), ]
# make sure final positions have been removed
PosDat$stim_number <- NumberStimuli(PosDat)
PosDat$raw_log_freq <- PosDat$log_freq
PosDat$log_freq <- PosDat$log_freq - mean(PosDat$log_freq) # centre frequency
OrigDataLen <- nrow(PosDat)
print(sprintf ("Original data has %d values", OrigDataLen))

## [1] "Original data has 4402 values"

if(RemoveFracPres){ # eliminate fractional preserved values?
  PosDat <- PosDat[(PosDat$preserved == 0) | (PosDat$preserved == 1),]
  DataLen<- nrow(PosDat)
  print(sprintf("Data without fractional preserved values has %d values", DataLen))
  print(sprintf("%d values eliminated.", OrigDataLen - DataLen))
}

```

```
## [1] "Data without fractional preserved values has 4227 values"
## [1] "175 values eliminated."
```

```
PosDat$CumPres <- CalcCumPres(PosDat)
PosDat$CumErr <- CalcCumErrFromPreserved(PosDat)
```

```
# plot distribution of complex onsets and codas
# across serial position in the stimuli -- do complexities concentrate
# on one part of the serial position curve?
# syll_component = 1 is coda
# syll_component = S is satellite
```

```
# get counts of syllable components in different serial positions
```

```
syll_comp_dist_N <- PosDat %>% mutate(pos_factor = as.factor(pos), syll_component_factor = as.factor(syll_component))
```

```
syll_comp_dist_perc <- syll_comp_dist_N %>% ungroup() %>%
  mutate(across(O:S, ~ . / total))
```

```
kable(syll_comp_dist_N)
```

pos_factor	O	P	V	1	S	total
1	549	35	130	NA	NA	714
2	66	NA	429	94	106	695
3	301	NA	166	204	14	685
4	297	NA	224	66	38	625
5	216	NA	208	65	37	526
6	202	1	128	69	19	419
7	169	NA	101	26	16	312
8	89	NA	53	23	4	169
9	73	NA	2	NA	7	82

```
kable(syll_comp_dist_perc)
```

pos_factor	O	P	V	1	S	total
1	0.7689076	0.0490196	0.1820728	NA	NA	714
2	0.0949640	NA	0.6172662	0.1352518	0.1525180	695
3	0.4394161	NA	0.2423358	0.2978102	0.0204380	685
4	0.4752000	NA	0.3584000	0.1056000	0.0608000	625
5	0.4106464	NA	0.3954373	0.1235741	0.0703422	526
6	0.4821002	0.0023866	0.3054893	0.1646778	0.0453461	419
7	0.5416667	NA	0.3237179	0.0833333	0.0512821	312
8	0.5266272	NA	0.3136095	0.1360947	0.0236686	169
9	0.8902439	NA	0.0243902	NA	0.0853659	82

```
# get percentages only from summary table
```

```
syll_comp_perc <- syll_comp_dist_perc[,2:(ncol(syll_comp_dist_perc)-1)]
```

```
syll_comp_perc$pos <- as.integer(as.character(syll_comp_dist_perc$pos_factor))
```

```
syll_comp_perc <- syll_comp_perc %>% rename("Coda" = "1", "Satellite" = "S")
```

```
# pivot to long format for plotting
```

```
syll_comp_plot_data <- syll_comp_perc %>% pivot_longer(cols=seq(1:(ncol(syll_comp_perc)-1)),
  names_to="syll_component", values_to="percent") %>% filter(syll_component %in% c("Coda", "Satellite"))
```

```
# plot satellite and coda occurrences across position

syll_comp_plot <- ggplot(syll_comp_plot_data,
                        aes(x=pos,y=percent,group=syll_component,
                           color=syll_component,
                           linetype = syll_component,
                           shape = syll_component)) +
  geom_line() + geom_point() + scale_color_manual(name="Syllable component", values = palette_values) +
syll_comp_plot <- syll_comp_plot + scale_y_continuous(name="Percent of segment types") + scale_linetype_manual(name="Syllable component", values = palette_linetypes) +
  scale_shape_discrete(name="Syllable component")
ggsave(paste0(FigDir, CurPat, "_", RunLabel, "_", CurTask, "_pos_of_syll_complexities_in_stimuli.png"), plot=syll_comp_plot)
```

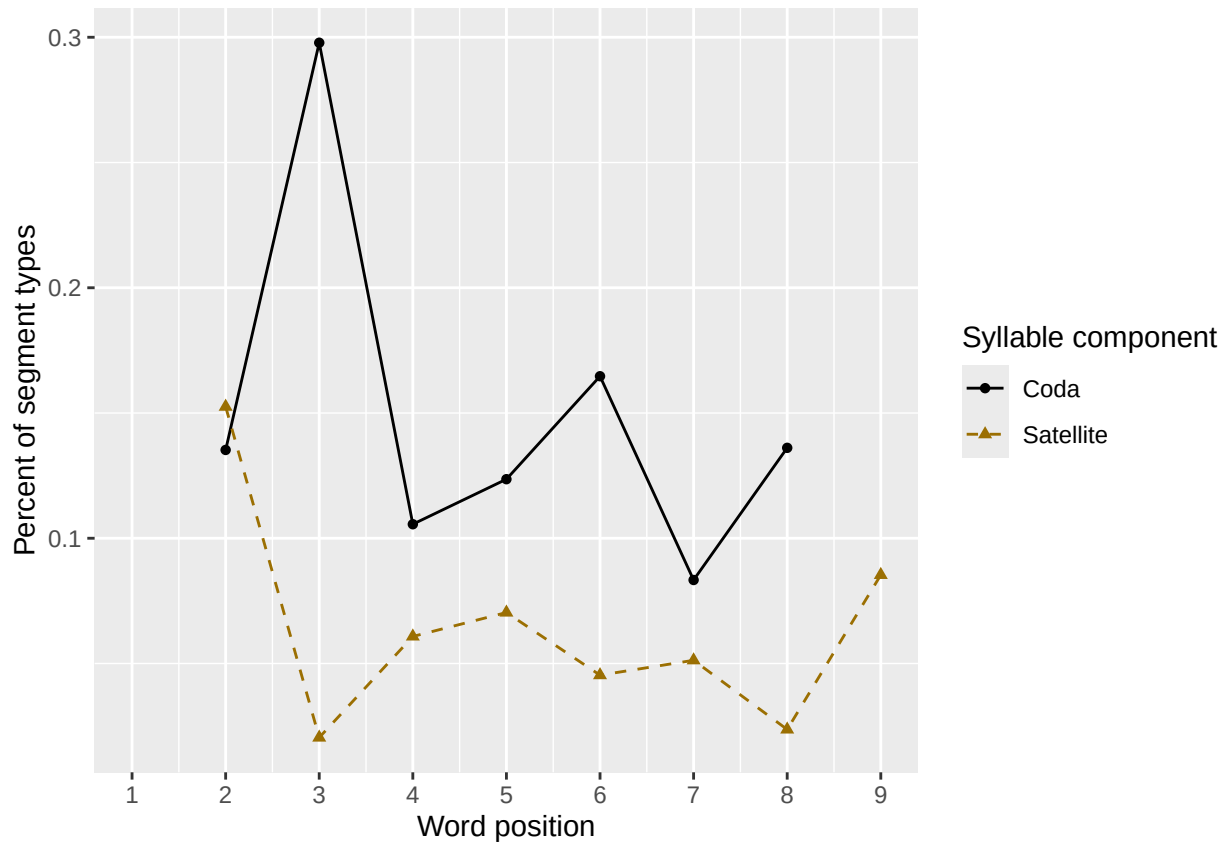
Warning: Removed 3 rows containing missing values or values outside the scale range (``geom_line()``).

Warning: Removed 3 rows containing missing values or values outside the scale range (``geom_point()``).

syll_comp_plot

```
## Warning: Removed 3 rows containing missing values or values outside the scale range (`geom_line()`).
```

```
## Removed 3 rows containing missing values or values outside the scale range (`geom_point()`).
```



```
# len/pos table
pos_len_summary <- PosDat %>% group_by(stimlen,pos) %>% summarise(preserved = mean(preserved))

## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.

min_preserved_raw <- min(as.numeric(unlist(pos_len_summary$preserved)))
max_preserved_raw <- max(as.numeric(unlist(pos_len_summary$preserved)))
preserved_range <- (max_preserved_raw - min_preserved_raw)
```

```

min_preserved <- min_preserved_raw - (0.1 * preserved_range)
max_preserved <- max_preserved_raw + (0.1 * preserved_range)
pos_len_table <- pos_len_summary %>% pivot_wider(names_from = pos, values_from = preserved)
write.csv(pos_len_table, paste0(TablesDir, CurPat, "_", RunLabel, "_", CurTask, "_preserved_percentages_by_len",
pos_len_table

```

```

## # A tibble: 7 x 10
## # Groups:   stimlen [7]
##   stimlen   `1`   `2`   `3`   `4`   `5`   `6`   `7`   `8`   `9`
##   <int> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl>
## 1      4 0.915 0.966 0.898 NA      NA      NA      NA      NA      NA
## 2      5 0.883 0.915 0.956 0.923 NA      NA      NA      NA      NA
## 3      6 0.958 0.930 0.901 0.918 0.935 NA      NA      NA      NA
## 4      7 0.911 0.918 0.889 0.855 0.917 0.873 NA      NA      NA
## 5      8 0.941 0.919 0.911 0.883 0.869 0.882 0.876 NA      NA
## 6      9 0.957 0.889 0.879 0.867 0.816 0.779 0.875 0.764 NA
## 7     10 0.953 0.936 0.899 0.848 0.792 0.747 0.810 0.9    0.805

```

```

# len/pos table
pos_len_N <- PosDat %>% group_by(stimlen, pos) %>% summarise(preserved = mean(preserved), N=n()) %>% dplyr::

```

`summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.

```

pos_len_N_table <- pos_len_N %>% pivot_wider(names_from = pos, values_from = N)
write.csv(pos_len_N_table, paste0(TablesDir, CurPat, "_", RunLabel, "_", CurTask,
"_N_by_len_position.csv"))
pos_len_N_table

```

```

## # A tibble: 7 x 10
## # Groups:   stimlen [7]
##   stimlen   `1`   `2`   `3`   `4`   `5`   `6`   `7`   `8`   `9`
##   <int> <int> <int> <int> <int> <int> <int> <int> <int> <int>
## 1      4    59    59    59    NA    NA    NA    NA    NA    NA
## 2      5    94    94    91    91    NA    NA    NA    NA    NA
## 3      6   118   115   111   110   108    NA    NA    NA    NA
## 4      7   112   110   108   110   109   110    NA    NA    NA
## 5      8   153   149   146   145   145   144   145    NA    NA
## 6      9    93    90    91    90    87    86    88    89    NA
## 7     10    85    78    79    79    77    79    79    80    82

```

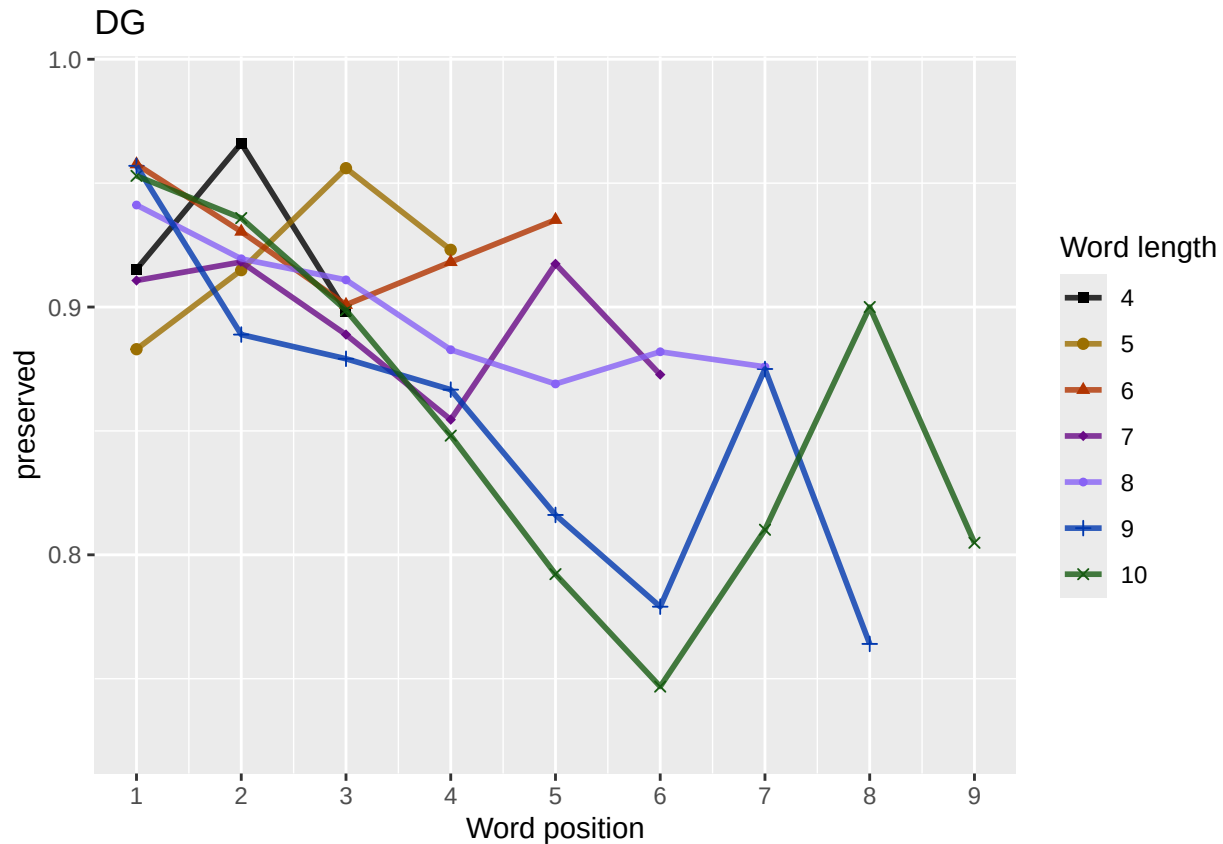
```

obs_linetypes <- c("solid", "solid", "solid", "solid",
"solid", "solid", "solid", "solid", "solid")
pred_linetypes <- "dashed"
pos_len_summary$stimlen <- factor(pos_len_summary$stimlen)
pos_len_summary$pos <- factor(pos_len_summary$pos)
# len_pos_plot <- ggplot(pos_len_summary, aes(x=pos, y=preserved, group=stimlen, shape=stimlen, color=stimlen))
len_pos_plot <- plot_len_pos_obs_predicted(PosDat,
paste0(CurPat),
NULL,
c(min_preserved, max_preserved),
palette_values,
shape_values,
obs_linetypes,
pred_linetypes)

```

`summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.

```
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## Scale for y is already present. Adding another scale for y, which will replace the existing scale.
ggsave(paste0(FigDir, CurPat, "_", RunLabel, "_", CurTask,
              "_percent_preserved_by_length_pos.png"),
       plot=len_pos_plot, device="png", unit="cm", width=15, height=11)
len_pos_plot
```



Length and position

```
# length and position

LPModelEquations<-c("preserved ~ 1",
                    "preserved ~ stimlen",
                    "preserved ~ pos",
                    "preserved ~ stimlen + pos",
                    "preserved ~ stimlen*pos",
                    "preserved ~ I(pos^2)+pos",
                    "preserved ~ stimlen + I(pos^2) + pos",
                    "preserved ~ stimlen * (I(pos^2) + pos)"
)

LPRes<-TestModels(LPModelEquations,PosDat)
```

```
## *****
## model index: 8
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##          data = PosDat)
```

```

##
## Coefficients:
##      (Intercept)      stimlen      I(pos^2)      pos  stimlen:I(pos^2)      stiml
##      0.91712      0.27304      -0.07756      0.95798      0.01343      -0
##
## Degrees of Freedom: 4226 Total (i.e. Null);  4221 Residual
## Null Deviance:      2934
## Residual Deviance: 2861 AIC: 2873
## log likelihood: -1430.48
## Nagelkerke R2: 0.03411296
## % pres/err predicted correctly: -813.3891
## % of predictable range [ (model-null)/(1-null) ]: 0.01792427
## *****
## model index: 7
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##      data = PosDat)
##
## Coefficients:
##      (Intercept)      stimlen      I(pos^2)      pos
##      3.65307      -0.09960      0.01738      -0.28296
##
## Degrees of Freedom: 4226 Total (i.e. Null);  4223 Residual
## Null Deviance:      2934
## Residual Deviance: 2875 AIC: 2883
## log likelihood: -1437.575
## Nagelkerke R2: 0.02750854
## % pres/err predicted correctly: -816.3899
## % of predictable range [ (model-null)/(1-null) ]: 0.01430563
## *****
## model index: 5
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##      data = PosDat)
##
## Coefficients:
##      (Intercept)      stimlen      pos  stimlen:pos
##      2.629721      -0.008879      0.076137      -0.023357
##
## Degrees of Freedom: 4226 Total (i.e. Null);  4223 Residual
## Null Deviance:      2934
## Residual Deviance: 2876 AIC: 2884
## log likelihood: -1437.941
## Nagelkerke R2: 0.02716653
## % pres/err predicted correctly: -816.3096
## % of predictable range [ (model-null)/(1-null) ]: 0.01440253
## *****
## model index: 4
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##      data = PosDat)
##
## Coefficients:
##      (Intercept)      stimlen      pos

```

```

##      3.30203      -0.08902      -0.12656
##
## Degrees of Freedom: 4226 Total (i.e. Null);  4224 Residual
## Null Deviance:      2934
## Residual Deviance: 2878  AIC: 2884
## log likelihood:  -1439.08
## Nagelkerke R2:  0.02610386
## % pres/err predicted correctly:  -816.8716
## % of predictable range [ (model-null)/(1-null) ]:  0.01372476
## *****
## model index:  3
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##      data = PosDat)
##
## Coefficients:
## (Intercept)      pos
##      2.7238      -0.1555
##
## Degrees of Freedom: 4226 Total (i.e. Null);  4225 Residual
## Null Deviance:      2934
## Residual Deviance: 2885  AIC: 2889
## log likelihood:  -1442.56
## Nagelkerke R2:  0.02285416
## % pres/err predicted correctly:  -818.4835
## % of predictable range [ (model-null)/(1-null) ]:  0.01178093
## *****
## model index:  6
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##      data = PosDat)
##
## Coefficients:
## (Intercept)      I(pos^2)      pos
##      2.92200      0.01223      -0.26786
##
## Degrees of Freedom: 4226 Total (i.e. Null);  4224 Residual
## Null Deviance:      2934
## Residual Deviance: 2884  AIC: 2890
## log likelihood:  -1441.793
## Nagelkerke R2:  0.02357081
## % pres/err predicted correctly:  -818.3749
## % of predictable range [ (model-null)/(1-null) ]:  0.0119119
## *****
## model index:  2
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##      data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen
##      3.3279      -0.1583
##
## Degrees of Freedom: 4226 Total (i.e. Null);  4225 Residual

```



```
## Null Deviance:      2934
## Residual Deviance: 2906 AIC: 2910
## log likelihood: -1452.882
## Nagelkerke R2: 0.01318384
## % pres/err predicted correctly: -822.811
## % of predictable range [ (model-null)/(1-null) ]: 0.006562388
## *****
## model index: 1
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)
## 2.088
##
## Degrees of Freedom: 4226 Total (i.e. Null); 4226 Residual
## Null Deviance:      2934
## Residual Deviance: 2934 AIC: 2936
## log likelihood: -1466.873
## Nagelkerke R2: 0
## % pres/err predicted correctly: -828.2529
## % of predictable range [ (model-null)/(1-null) ]: 0
## *****
```

```
BestLPModel<-LPRes$ModelResult[[1]]
BestLPModelFormula<-LPRes$Model[[1]]

LPAICSummary<-data.frame(Model=LPRes$Model,
                          AIC=LPRes$AIC,
                          row.names = LPRes$Model)
LPAICSummary$DeltaAIC<-LPAICSummary$AIC-LPAICSummary$AIC[1]
LPAICSummary$AICexp<-exp(-0.5*LPAICSummary$DeltaAIC)
LPAICSummary$AICwt<-LPAICSummary$AICexp/sum(LPAICSummary$AICexp)
LPAICSummary$NagR2<-LPRes$NagR2

LPAICSummary <- merge(LPAICSummary,LPRes$CoefficientValues, by='row.names',sort=FALSE)
LPAICSummary <- subset(LPAICSummary, select = -c(Row.names))

write.csv(LPAICSummary,paste0(TablesDir,CurPat,"_",RunLabel,"_",CurTask,
                              "_len_pos_model_summary.csv"),row.names = FALSE)

kable(LPAICSummary)
```

Model	AIC	DeltaAIC	AICexp	AICwt	NagR2	(Intercept)	stimlen	pos	stimlen:I(pos)	I(pos^2)	stimlen:I(pos^2)
preserved ~ stimlen * (I(pos^2) + pos)	2872.959	0.000001	0.000000	0.008558	0.020341	1.309171	1160.273042	229579760	-	-	0.0134346
preserved ~ stimlen + I(pos^2) + pos	2883.140	0.18983	0.006127	0.006039	0.027508	1.565307	30	730	-	-	NA
preserved ~ stimlen * pos	2883.883	0.92335	0.004246	0.004185	0.027162	1.656297	207	0.0761374	-	NA	NA
preserved ~ stimlen + pos	2884.161	1.20170	0.003694	0.003640	0.026103	1.330202	274	-	-	NA	NA

Model	AIC	DeltaAIC	AICexp	AICwt	NagR2	(Intercept)	stimlen	pos	stimlen:I(pos^2)	I(pos^2)	stimlen:I(pos^2)
preserved ~ pos	2889.1216	16.1614	0.0003095	0.0003095	0.0022854	27237781	NA	-	NA	NA	NA
								0.1554804			
preserved ~ I(pos^2) + pos	2889.5876	6.6275	7.0002451	0.0002406	0.0023570	289220029	NA	-	NA	0.0122337	NA
								0.2678608			
preserved ~ stimlen	2909.7636	8.0423	0.0000000	0.0000000	0.0013183	33278726	-	NA	NA	NA	NA
								0.1583129			
preserved ~ 1	2935.7452	7.8580	0.0000000	0.0000000	0.0000000	200882545	NA	NA	NA	NA	NA

```
print(BestLPModelFormula)
```

```
## [1] "preserved ~ stimlen * (I(pos^2) + pos)"
```

```
print(BestLPModel)
```

```
##
```

```
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
```

```
## data = PosDat)
```

```
##
```

```
## Coefficients:
```

```
## (Intercept) stimlen I(pos^2) pos stimlen:I(pos^2) stimlen:I(pos^2)
```

```
## 0.91712 0.27304 -0.07756 0.95798 0.01343 -0.01343
```

```
##
```

```
## Degrees of Freedom: 4226 Total (i.e. Null); 4221 Residual
```

```
## Null Deviance: 2934
```

```
## Residual Deviance: 2861 AIC: 2873
```

```
PosDat$LPFitted<-fitted(BestLPModel)
```

```
fitted_len_pos_plot <- plot_len_pos_obs_predicted(PosDat, PosDat$patient[1],
NULL,palette_values=palette_values)
```

```
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
```

```
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
```

```
# len/pos table
```

```
fitted_pos_len_summary <- PosDat %>% group_by(stimlen,pos) %>% summarise(fitted = mean(LPfitted))
```

```
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
```

```
fitted_pos_len_table <- fitted_pos_len_summary %>% pivot_wider(names_from = pos, values_from = fitted)
```

```
fitted_pos_len_table
```

```
## # A tibble: 7 x 10
```

```
## # Groups: stimlen [7]
```

```
## stimlen `1` `2` `3` `4` `5` `6` `7` `8` `9`
```

```
## <int> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl>
```

```
## 1 4 0.906 0.923 0.934 NA NA NA NA NA NA
```

```
## 2 5 0.916 0.922 0.926 0.929 NA NA NA NA NA
```

```
## 3 6 0.924 0.921 0.918 0.915 0.913 NA NA NA NA
```

```
## 4 7 0.932 0.920 0.909 0.900 0.893 0.889 NA NA NA
```

```
## 5 8 0.939 0.919 0.899 0.882 0.868 0.860 0.860 NA NA
```

```
## 6 9 0.946 0.919 0.889 0.860 0.839 0.827 0.827 0.839 NA
```

```
## 7 10 0.951 0.918 0.877 0.836 0.804 0.787 0.788 0.807 0.841
```

```
fitted_pos_len_summary$stimlen<-factor(fitted_pos_len_summary$stimlen)
```

```
fitted_pos_len_summary$pos<-factor(fitted_pos_len_summary$pos)
```

```

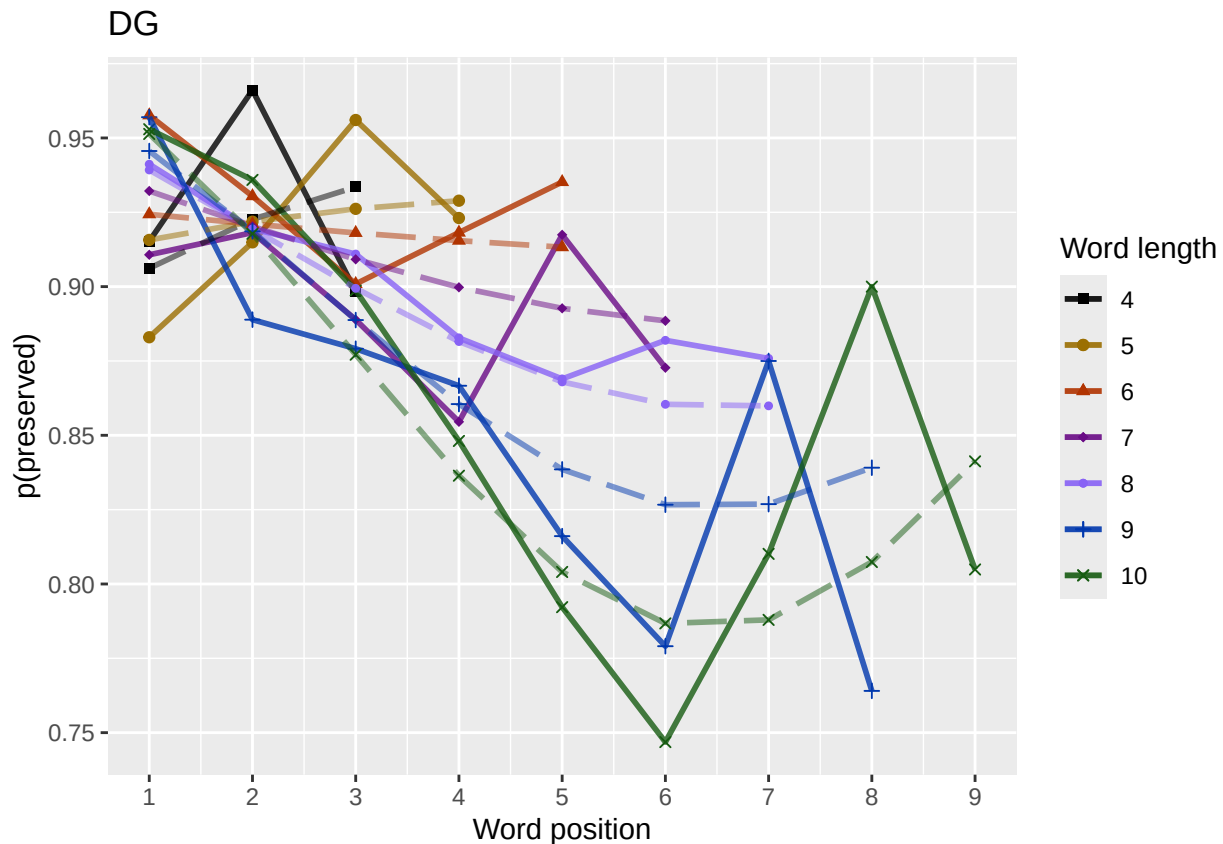
# fitted_len_pos_plot <- ggplot(pos_len_summary, aes(x=pos, y=preserved, group=stimlen, shape=stimlen, color=stimlen))
# fitted_len_pos_plot <- fitted_len_pos_plot + geom_line(data=fitted_pos_len_summary, aes(x=pos, y=fitted, group=stimlen, shape=stimlen, color=stimlen))
# geom_point(data=fitted_pos_len_summary, aes(x=pos, y=fitted, group=stimlen, shape=stimlen)) + ggtitle(paste0("Patient", patient_id))

fitted_len_pos_plot <- plot_len_pos_obs_predicted(PosDat,
  paste0(PosDat$patient[1]),
  "LPFitted",
  NULL,
  palette_values,
  shape_values,
  obs_linetypes,
  pred_linetypes = c("longdash")
)

## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.

ggsave(paste0(FigDir, CurPat, "_", RunLabel, "_", CurTask, "_percent_preserved_by_length_pos_wfit.png"), plot=fitted_len_pos_plot

```



length and position without fragments to see if this changes position² influence

```

# first number responses, then count resp with fragments - below we will eliminate fragments
# and re-run models

# number responses
resp_num <- 0

```

```

prev_pos<-9999 # big number to initialize (so first position is smaller)
resp_num_array <- integer(length = nrow(PosDat))
for(i in seq(1,nrow(PosDat))){
  if(PosDat$pos[i] <= prev_pos){ # equal for resp length 1
    resp_num <- resp_num + 1
  }
  resp_num_array[i]<-resp_num
  prev_pos <- PosDat$pos[i]
}
PosDat$resp_num <- resp_num_array

# count responses with fragments
resp_with_frag <- PosDat %>% group_by(resp_num) %>% summarise(frag = as.logical(sum(fragment)))
num_frag <- resp_with_frag %>% summarise(frag_sum = sum(frag), N=n())
num_frag

## # A tibble: 1 x 2
##   frag_sum      N
##   <int> <int>
## 1      9    715

num_frag$percent_with_frag[1] <- (num_frag$frag_sum[1] / num_frag$N[1])*100

## Warning: Unknown or uninitialised column: `percent_with_frag`.

print(sprintf("The number of responses with fragments was %d / %d = %.2f percent",
  num_frag$frag_sum[1],num_frag$N[1],num_frag$percent_with_frag[1]))

## [1] "The number of responses with fragments was 9 / 715 = 1.26 percent"

write.csv(num_frag,paste0(TablesDir,CurPat,"_",RunLabel,"_",
  CurTask,"_percent_fragments.csv"),row.names = FALSE)

# length and position models for data without fragments
NoFragData <- PosDat %>% filter(fragment != 1)

NoFrag_LPRes<-TestModels(LPModelEquations,NoFragData)

## *****
## model index: 8
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##   data = PosDat)
##
## Coefficients:
##   (Intercept)      stimlen      I(pos^2)      pos  stimlen:I(pos^2)      stimlen
##   0.77537      0.29597      -0.07865      1.03469      0.01437      -0.
##
## Degrees of Freedom: 4208 Total (i.e. Null); 4203 Residual
## Null Deviance: 2854
## Residual Deviance: 2789 AIC: 2801
## log likelihood: -1394.301
## Nagelkerke R2: 0.03118707
## % pres/err predicted correctly: -786.5939
## % of predictable range [ (model-null)/(1-null) ]: 0.01628359
## *****

```

```

## model index: 7
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen      I(pos^2)          pos
##      3.72913      -0.10735      0.02179      -0.30129
##
## Degrees of Freedom: 4208 Total (i.e. Null); 4205 Residual
## Null Deviance:      2854
## Residual Deviance: 2805 AIC: 2813
## log likelihood: -1402.613
## Nagelkerke R2: 0.02327329
## % pres/err predicted correctly: -790.0966
## % of predictable range [ (model-null)/(1-null) ]: 0.01190863
## *****
## model index: 5
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen          pos stimlen:pos
##      2.60406      -0.01166      0.10592      -0.02455
##
## Degrees of Freedom: 4208 Total (i.e. Null); 4205 Residual
## Null Deviance:      2854
## Residual Deviance: 2807 AIC: 2815
## log likelihood: -1403.641
## Nagelkerke R2: 0.0222919
## % pres/err predicted correctly: -790.4407
## % of predictable range [ (model-null)/(1-null) ]: 0.01147891
## *****
## model index: 4
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen          pos
##      3.2996      -0.0946      -0.1070
##
## Degrees of Freedom: 4208 Total (i.e. Null); 4206 Residual
## Null Deviance:      2854
## Residual Deviance: 2810 AIC: 2816
## log likelihood: -1404.85
## Nagelkerke R2: 0.02113785
## % pres/err predicted correctly: -790.959
## % of predictable range [ (model-null)/(1-null) ]: 0.01083154
## *****
## model index: 6
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",

```

```

##      data = PosDat)
##
## Coefficients:
## (Intercept)      I(pos^2)          pos
##      2.93967      0.01622      -0.28483
##
## Degrees of Freedom: 4208 Total (i.e. Null);  4206 Residual
## Null Deviance:      2854
## Residual Deviance: 2815  AIC: 2821
## log likelihood:  -1407.444
## Nagelkerke R2:  0.0186589
## % pres/err predicted correctly:  -792.326
## % of predictable range [ (model-null)/(1-null) ]:  0.009124124
## *****
## model index:  3
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##      data = PosDat)
##
## Coefficients:
## (Intercept)          pos
##      2.6825      -0.1373
##
## Degrees of Freedom: 4208 Total (i.e. Null);  4207 Residual
## Null Deviance:      2854
## Residual Deviance: 2817  AIC: 2821
## log likelihood:  -1408.717
## Nagelkerke R2:  0.01744072
## % pres/err predicted correctly:  -792.6576
## % of predictable range [ (model-null)/(1-null) ]:  0.008709887
## *****
## model index:  2
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##      data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen
##      3.316      -0.152
##
## Degrees of Freedom: 4208 Total (i.e. Null);  4207 Residual
## Null Deviance:      2854
## Residual Deviance: 2829  AIC: 2833
## log likelihood:  -1414.373
## Nagelkerke R2:  0.01202232
## % pres/err predicted correctly:  -794.9026
## % of predictable range [ (model-null)/(1-null) ]:  0.005905908
## *****
## model index:  1
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##      data = PosDat)
##
## Coefficients:

```

```

## (Intercept)
##      2.128
##
## Degrees of Freedom: 4208 Total (i.e. Null);  4208 Residual
## Null Deviance:      2854
## Residual Deviance: 2854  AIC: 2856
## log likelihood:  -1426.868
## Nagelkerke R2:  0
## % pres/err predicted correctly:  -799.631
## % of predictable range [ (model-null)/(1-null) ]:  0
## *****

NoFragBestLPModel<-NoFrag_LPRes$ModelResult[[1]]
NoFragBestLPModelFormula<-NoFrag_LPRes$Model[[1]]

NoFragLPAICSummary<-data.frame(Model=NoFrag_LPRes$Model,
                                AIC=NoFrag_LPRes$AIC,
                                row.names = NoFrag_LPRes$Model)
NoFragLPAICSummary$DeltaAIC<-NoFragLPAICSummary$AIC-NoFragLPAICSummary$AIC[1]
NoFragLPAICSummary$AICexp<-exp(-0.5*NoFragLPAICSummary$DeltaAIC)
NoFragLPAICSummary$AICwt<-NoFragLPAICSummary$AICexp/sum(NoFragLPAICSummary$AICexp)
NoFragLPAICSummary$NagR2<-NoFrag_LPRes$NagR2

NoFragLPAICSummary <- merge(NoFragLPAICSummary,NoFrag_LPRes$CoefficientValues, by='row.names',sort=FALSE)
NoFragLPAICSummary <- subset(NoFragLPAICSummary, select = -c(Row.names))

write.csv(NoFragLPAICSummary,paste0(TablesDir,
                                    CurPat,"_",RunLabel,"_",
                                    CurTask,
                                    "_no_fragments_len_pos_model_summary.csv"),
          row.names = FALSE)
kable(NoFragLPAICSummary)

```

Model	AIC	DeltaAIC	AICexp	AICwt	NagR2	(Intercept)	stimlen	pos	stimlen:I(pos	stimlen:I(pos^2)	stimlen:I(pos^2)
preserved ~ stimlen * (I(pos^2) + pos)	2800.602	0.000001	0.000000	0.0969493	0.311871	7753697	220346925	-	-	0.0143665	
preserved ~ stimlen + I(pos^2) + pos	2813.225	12.62339	0.001815	0.0001809	0.4023273	37291338	-	-	NA	0.0217899	NA
preserved ~ stimlen * pos	2815.282	14.68029	0.000649	0.0000647	0.0222929	6040559	-	0.1059201	-	NA	NA
preserved ~ stimlen + pos	2815.700	5.09780	0.000526	0.0005251	0.2113382	2996431	-	-	NA	NA	NA
preserved ~ I(pos^2) + pos	2820.882	10.28600	0.000039	0.0000392	0.186589	396672	NA	-	NA	0.0162250	NA
preserved ~ pos	2821.432	10.83310	0.000029	0.0000298	0.174407	6824908	NA	-	NA	NA	NA
preserved ~ stimlen	2832.746	12.14430	0.000000	0.0000001	0.1020233	162331	-	NA	NA	NA	NA
preserved ~ 1	2855.735	55.13336	0.000000	0.0000000	0.0000000	01276469	NA	NA	NA	NA	NA

```
# plot no fragment data
```

```
NoFragData$LPFitted<-fitted(NoFragBestLPModel)
nofrag_obs_len_pos_plot <- plot_len_pos_obs_predicted(NoFragData, NoFragData$patient[1],
  NULL,palette_values=palette_values)
```

```
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
```

```
# len/pos table
```

```
nofrag_fitted_pos_len_summary <- NoFragData %>% group_by(stimlen,pos) %>% summarise(fitted = mean(LPFit
```

```
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
```

```
nofrag_fitted_pos_len_table <- nofrag_fitted_pos_len_summary %>% pivot_wider(names_from = pos, values_f
nofrag_fitted_pos_len_table
```

```
## # A tibble: 7 x 10
## # Groups:   stimlen [7]
##   stimlen   `1`   `2`   `3`   `4`   `5`   `6`   `7`   `8`   `9`
##   <int> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl>
## 1      4 0.904 0.923 0.936 NA      NA      NA      NA      NA      NA
## 2      5 0.915 0.922 0.928 0.933 NA      NA      NA      NA      NA
## 3      6 0.924 0.921 0.920 0.919 0.919 NA      NA      NA      NA
## 4      7 0.932 0.920 0.910 0.902 0.898 0.898 NA      NA      NA
## 5      8 0.940 0.919 0.899 0.883 0.872 0.869 0.874 NA      NA
## 6      9 0.947 0.919 0.888 0.860 0.840 0.833 0.840 0.859 NA
## 7     10 0.953 0.918 0.875 0.833 0.802 0.790 0.799 0.827 0.867
```

```
fitted_pos_len_summary$stimlen<-factor(nofrag_fitted_pos_len_summary$stimlen)
```

```
fitted_pos_len_summary$pos<-factor(nofrag_fitted_pos_len_summary$pos)
```

```
# fitted_len_pos_plot <- ggplot(pos_len_summary,aes(x=pos,y=preserved,group=stimlen,shape=stimlen,color
```

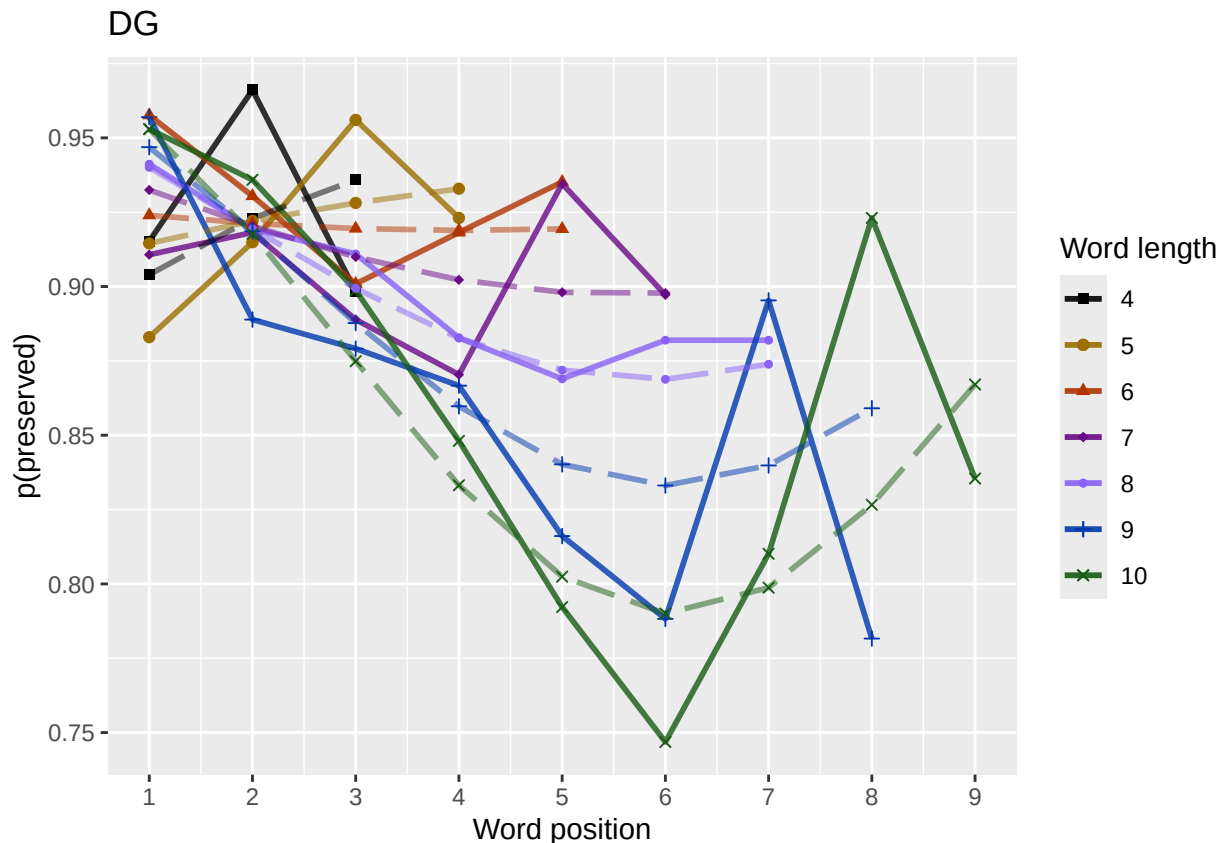
```
# fitted_len_pos_plot <- fitted_len_pos_plot + geom_line(data=fitted_pos_len_summary, aes(x=pos,y=fitted
```

```
# geom_point(data=fitted_pos_len_summary,aes(x=pos,y=fitted,group=stimlen,shape=stimlen)) + ggtitle(pas
```

```
nofrag_fitted_len_pos_plot <- plot_len_pos_obs_predicted(NoFragData,
  paste0(NoFragData$patient[1]),
  "LPFitted",
  NULL,
  palette_values,
  shape_values,
  obs_linetypes,
  pred_linetypes = c("longdash")
)
```

```
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
```

```
ggsave(paste0(FigDir,CurPat,"_",RunLabel,"_",CurTask,
  "no_fragments_percent_preserved_by_length_pos_wfit.png"),
  plot=nofrag_fitted_len_pos_plot,
  device="png",unit="cm",
  width=15,height=11)
nofrag_fitted_len_pos_plot
```

back to full data

```
results_report_DF <- AddReportLine(results_report_DF,"min preserved",min_preserved)
results_report_DF <- AddReportLine(results_report_DF,"max preserved",max_preserved)
print(sprintf("Min/max preserved range: %.2f - %.2f",min_preserved,max_preserved))
```

```
## [1] "Min/max preserved range: 0.72 - 0.99"
```

```
# find the average difference between lengths and the average difference
# between positions taking the other factor into account
```

```
# choose fitted or raw -- we use fitted values because they are smoothed averages
# that take into account the influence of the other variable
```

```
table_to_use <- fitted_pos_len_table
```

```
# take away column with length information
```

```
table_to_use <- table_to_use[,2:ncol(table_to_use)]
```

```
# take averages for positions
```

```
# don't want downward estimates influenced by return upward of U
```

```
# therefore, for downward influence, use only the values before the min
```

```
# take the difference between each value (differences between position proportion correct) **NOTE** pro
```

```
# average the difference in probabilities
```

```
# in case min is close to the end or we are not using a min (for non-U-shaped)
```

```
# functions, we need to get differences _first_ and then average those (e.g.
```

```
# if there is an effect of length and we just average position data,
```

```
# later positions) will have a lower average (because of the length effect)
```

```
# even if all position functions go upward
```

```

table_pos_diffs <- t(diff(t(as.matrix(table_to_use))))
ncol_pos_diff_matrix <- ncol(table_pos_diffs)
first_col_mean <- mean(as.numeric(unlist(table_pos_diffs[,1])))
potential_u_shape <- 0
if(first_col_mean < 0){ # downward, so potential U-shape
  # take only negative values from the table
  filtered_table_pos_diffs<-table_pos_diffs
  filtered_table_pos_diffs[table_pos_diffs >= 0] <- NA
  potential_u_shape <- 1
}else{
  # upward - use the positive values
  # take only negative values from the table
  filtered_table_pos_diffs<-table_pos_diffs
  filtered_table_pos_diffs[table_pos_diffs <= 0] <- NA
}
average_pos_diffs <- apply(filtered_table_pos_diffs,2,mean,na.rm=TRUE)
OA_mean_pos_diff <- mean(average_pos_diffs,na.rm=TRUE)

table_len_diffs <- diff(as.matrix(table_to_use))
average_len_diffs <- apply(table_len_diffs,1,mean,na.rm=TRUE)
OA_mean_len_diff <- mean(average_len_diffs,na.rm=TRUE)
CurrentLabel<-"mean change in probability for each additional length: "
print(CurrentLabel)

## [1] "mean change in probability for each additional length: "
print(OA_mean_len_diff)

## [1] -0.0104132
results_report_DF <- AddReportLine(results_report_DF, CurrentLabel,OA_mean_len_diff)

CurrentLabel<-"mean change in probability for each additional position (excluding U-shape): "
print(CurrentLabel)

## [1] "mean change in probability for each additional position (excluding U-shape): "
print(OA_mean_pos_diff)

## [1] -0.0143323
results_report_DF <- AddReportLine(results_report_DF, CurrentLabel,OA_mean_pos_diff)

if(potential_u_shape){ # downward, so potential U-shape
  # take only positive values from the table
  filtered_pos_upward_u<-table_pos_diffs
  filtered_pos_upward_u[table_pos_diffs <= 0] <- NA
  average_pos_u_diffs <- apply(filtered_pos_upward_u,
    2,mean,na.rm=TRUE)
  OA_mean_pos_u_diff <- mean(average_pos_u_diffs,na.rm=TRUE)
  if(is.nan(OA_mean_pos_u_diff) | (OA_mean_pos_u_diff <= 0)){
    print("No U-shape in this participant")
    results_report_DF <- AddReportLine(results_report_DF, "U-shape", 0)
    potential_u_shape <- FALSE
  }else{
    results_report_DF <- AddReportLine(results_report_DF, "U-shape", 1)
  }
}

```

```

    CurrentLabel<-"Average upward change after U minimum"
    print(CurrentLabel)
    print(OA_mean_pos_u_diff)
    results_report_DF <- AddReportLine(results_report_DF, CurrentLabel, OA_mean_pos_u_diff)

    CurrentLabel<-"Proportion of average downward change"
    prop_ave_down <- abs(OA_mean_pos_u_diff/OA_mean_pos_diff)
    results_report_DF <- AddReportLine(results_report_DF, CurrentLabel, prop_ave_down)
  }
}else{
  print("No U-shape in this participant")
  results_report_DF <- AddReportLine(results_report_DF, "U-shape", 0)
}

## [1] "Average upward change after U minimum"
## [1] 0.0120053

if(potential_u_shape){
  n_rows<-nrow(table_to_use)
  downward_dist <- numeric(n_rows)
  upward_dist <- numeric(n_rows)
  for(i in seq(1,n_rows)){
    current_row <- as.numeric(unlist(table_to_use[i,1:9]))
    current_row <- current_row[!is.na(current_row)]
    current_row_len <- length(current_row)
    row_min <- min(current_row,na.rm=TRUE)
    min_pos <- which(current_row == row_min)
    left_max <- max(current_row[1:min_pos],na.rm=TRUE)
    right_max <- max(current_row[min_pos:current_row_len])
    left_diff <- left_max - row_min
    right_diff <- right_max - row_min
    downward_dist[i]<-left_diff
    upward_dist[i]<-right_diff
  }
  print("differences from left max to min for each row: ")
  print(downward_dist)
  print("differences from min to right max for each row: ")
  print(upward_dist)

  # Use the max return upward (min of upward_dist) and then the
  # downward distance from the left for the same row

  print(" ")
  biggest_return_upward_row <- which(upward_dist == max(upward_dist))
  CurrentLabel <- "row with biggest return upward"
  print(CurrentLabel)
  print(biggest_return_upward_row)
  results_report_DF <- AddReportLine(results_report_DF, CurrentLabel, biggest_return_upward_row)

  print(" ")
  CurrentLabel<-"downward distance for row with the largest upward value"
  print(CurrentLabel)
  print(downward_dist[biggest_return_upward_row])
  results_report_DF <- AddReportLine(results_report_DF, CurrentLabel, downward_dist[biggest_return_upward_row])
}

```

```

CurrentLabel<-"return upward value"
print(upward_dist[biggest_return_upward_row])
results_report_DF <- AddReportLine(results_report_DF,
                                   CurrentLabel,
                                   upward_dist[biggest_return_upward_row])

print(" ")
# percentage return upward
percentage_return_upward <- upward_dist[biggest_return_upward_row]/downward_dist[biggest_return_upward_row]
CurrentLabel <- "Return upward as a proportion of the downward distance:"
print(CurrentLabel)
print(percentage_return_upward)
results_report_DF <- AddReportLine(results_report_DF, CurrentLabel,
                                   percentage_return_upward)
}else{
  print("no U-shape in this participant")
}

## [1] "differences from left max to min for each row: "
## [1] 0.00000000 0.00000000 0.01107570 0.04366298 0.07933347 0.11895615 0.16461514
## [1] "differences from min to right max for each row: "
## [1] 0.02733689 0.01319473 0.00000000 0.00000000 0.00000000 0.01245053 0.05450366
## [1] " "
## [1] "row with biggest return upward"
## [1] 7
## [1] " "
## [1] "downward distance for row with the largest upward value"
## [1] 0.1646151
## [1] 0.05450366
## [1] " "
## [1] "Return upward as a proportion of the downward distance:"
## [1] 0.3310975

FLPModelEquations<-c(
  # models with log frequency, but no three-way interactions (due to difficulty interpreting and small
  "preserved ~ stimlen*log_freq",
  "preserved ~ stimlen+log_freq",
  "preserved ~ pos*log_freq",
  "preserved ~ pos+log_freq",
  "preserved ~ stimlen*log_freq + pos*log_freq",
  "preserved ~ stimlen*log_freq + pos",
  "preserved ~ stimlen + pos*log_freq",
  "preserved ~ stimlen + pos + log_freq",
  "preserved ~ (I(pos^2)+pos)*log_freq",
  "preserved ~ stimlen*log_freq + (I(pos^2) + pos)*log_freq",
  "preserved ~ stimlen*log_freq + I(pos^2) + pos",
  "preserved ~ stimlen + (I(pos^2) + pos)*log_freq",
  "preserved ~ stimlen + I(pos^2) + pos + log_freq",

  # models without frequency
  "preserved ~ 1",
  "preserved ~ stimlen",
  "preserved ~ pos",
  "preserved ~ stimlen + pos",

```

```

    "preserved ~ stimlen*pos",
    "preserved ~ I(pos^2)+pos",
    "preserved ~ stimlen + I(pos^2) + pos",
    "preserved ~ stimlen * (I(pos^2) + pos)"
)

FLPRes<-TestModels(FLPModelEquations,PosDat)

## *****
## model index:  21
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##         data = PosDat)
##
## Coefficients:
##      (Intercept)      stimlen      I(pos^2)      pos  stimlen:I(pos^2)      stimlen:I(pos^2)
##      0.91712      0.27304      -0.07756      0.95798      0.01343      -0.01343
##
## Degrees of Freedom: 4226 Total (i.e. Null);  4221 Residual
## Null Deviance:      2934
## Residual Deviance: 2861  AIC: 2873
## log likelihood:  -1430.48
## Nagelkerke R2:  0.03411296
## % pres/err predicted correctly:  -813.3891
## % of predictable range [ (model-null)/(1-null) ]:  0.01792427
## *****
## model index:  12
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##         data = PosDat)
##
## Coefficients:
##      (Intercept)      stimlen      I(pos^2)      pos      log_freq  I(pos^2)
##      3.46955      -0.07890      0.01390     -0.25801     -0.13466
##
## Degrees of Freedom: 4226 Total (i.e. Null);  4220 Residual
## Null Deviance:      2934
## Residual Deviance: 2865  AIC: 2879
## log likelihood:  -1432.521
## Nagelkerke R2:  0.03221466
## % pres/err predicted correctly:  -814.0086
## % of predictable range [ (model-null)/(1-null) ]:  0.01717724
## *****
## model index:  10
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##         data = PosDat)
##
## Coefficients:
##      (Intercept)      stimlen      log_freq      I(pos^2)      pos      stimlen:I(pos^2)
##      3.449250      -0.078756     -0.003102      0.014085     -0.256985
##      log_freq:pos
##      0.113011
##

```

```

## Degrees of Freedom: 4226 Total (i.e. Null); 4219 Residual
## Null Deviance: 2934
## Residual Deviance: 2864 AIC: 2880
## log likelihood: -1432.04
## Nagelkerke R2: 0.03266255
## % pres/err predicted correctly: -813.9324
## % of predictable range [ (model-null)/(1-null) ]: 0.01726919
## *****
## model index: 13
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept) stimlen I(pos^2) pos log_freq
## 3.52690 -0.08299 0.01717 -0.28136 0.05321
##
## Degrees of Freedom: 4226 Total (i.e. Null); 4222 Residual
## Null Deviance: 2934
## Residual Deviance: 2871 AIC: 2881
## log likelihood: -1435.621
## Nagelkerke R2: 0.02932941
## % pres/err predicted correctly: -815.6805
## % of predictable range [ (model-null)/(1-null) ]: 0.01516117
## *****
## model index: 11
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept) stimlen log_freq I(pos^2) pos stimlen:log_freq
## 3.47715 -0.08000 0.23782 0.01656 -0.27574 -0.013347
##
## Degrees of Freedom: 4226 Total (i.e. Null); 4221 Residual
## Null Deviance: 2934
## Residual Deviance: 2869 AIC: 2881
## log likelihood: -1434.657
## Nagelkerke R2: 0.03022714
## % pres/err predicted correctly: -815.3764
## % of predictable range [ (model-null)/(1-null) ]: 0.01552788
## *****
## model index: 9
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept) I(pos^2) pos log_freq I(pos^2):log_freq
## 2.886715 0.009708 -0.244486 -0.109666 -0.013347
##
## Degrees of Freedom: 4226 Total (i.e. Null); 4221 Residual
## Null Deviance: 2934
## Residual Deviance: 2870 AIC: 2882

```

```

## log likelihood: -1434.948
## Nagelkerke R2: 0.02995581
## % pres/err predicted correctly: -815.1427
## % of predictable range [ (model-null)/(1-null) ]: 0.01580961
## *****
## model index: 6
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept) stimlen log_freq pos stimlen:log_freq
## 3.13988 -0.06957 0.24650 -0.12690 -0.02399
##
## Degrees of Freedom: 4226 Total (i.e. Null); 4222 Residual
## Null Deviance: 2934
## Residual Deviance: 2872 AIC: 2882
## log likelihood: -1436.015
## Nagelkerke R2: 0.02896191
## % pres/err predicted correctly: -815.8265
## % of predictable range [ (model-null)/(1-null) ]: 0.01498507
## *****
## model index: 8
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept) stimlen pos log_freq
## 3.17927 -0.07238 -0.12697 0.05378
##
## Degrees of Freedom: 4226 Total (i.e. Null); 4223 Residual
## Null Deviance: 2934
## Residual Deviance: 2874 AIC: 2882
## log likelihood: -1437.085
## Nagelkerke R2: 0.02796459
## % pres/err predicted correctly: -816.2007
## % of predictable range [ (model-null)/(1-null) ]: 0.01453386
## *****
## model index: 20
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept) stimlen I(pos^2) pos
## 3.65307 -0.09960 0.01738 -0.28296
##
## Degrees of Freedom: 4226 Total (i.e. Null); 4223 Residual
## Null Deviance: 2934
## Residual Deviance: 2875 AIC: 2883
## log likelihood: -1437.575
## Nagelkerke R2: 0.02750854
## % pres/err predicted correctly: -816.3899

```

```

## % of predictable range [ (model-null)/(1-null) ]: 0.01430563
## *****
## model index: 7
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen          pos      log_freq pos:log_freq
##    3.170428    -0.070060    -0.129751    0.082348    -0.006513
##
## Degrees of Freedom: 4226 Total (i.e. Null); 4222 Residual
## Null Deviance:      2934
## Residual Deviance: 2874 AIC: 2884
## log likelihood: -1436.932
## Nagelkerke R2: 0.02810754
## % pres/err predicted correctly: -816.1371
## % of predictable range [ (model-null)/(1-null) ]: 0.01461045
## *****
## model index: 18
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen          pos stimlen:pos
##    2.629721    -0.008879    0.076137    -0.023357
##
## Degrees of Freedom: 4226 Total (i.e. Null); 4223 Residual
## Null Deviance:      2934
## Residual Deviance: 2876 AIC: 2884
## log likelihood: -1437.941
## Nagelkerke R2: 0.02716653
## % pres/err predicted correctly: -816.3096
## % of predictable range [ (model-null)/(1-null) ]: 0.01440253
## *****
## model index: 5
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen      log_freq          pos stimlen:log_freq      log_fr
##    3.1399859    -0.0697540    0.2467148    -0.1265854    -0.0244564    0.0
##
## Degrees of Freedom: 4226 Total (i.e. Null); 4221 Residual
## Null Deviance:      2934
## Residual Deviance: 2872 AIC: 2884
## log likelihood: -1436.013
## Nagelkerke R2: 0.02896372
## % pres/err predicted correctly: -815.8283
## % of predictable range [ (model-null)/(1-null) ]: 0.01498289
## *****
## model index: 17

```



```

##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##       data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen          pos
##      3.30203      -0.08902      -0.12656
##
## Degrees of Freedom: 4226 Total (i.e. Null);  4224 Residual
## Null Deviance:      2934
## Residual Deviance: 2878  AIC: 2884
## log likelihood:  -1439.08
## Nagelkerke R2:  0.02610386
## % pres/err predicted correctly:  -816.8716
## % of predictable range [ (model-null)/(1-null) ]:  0.01372476
## *****
## model index:  4
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##       data = PosDat)
##
## Coefficients:
## (Intercept)          pos      log_freq
##      2.70384      -0.14889      0.06733
##
## Degrees of Freedom: 4226 Total (i.e. Null);  4224 Residual
## Null Deviance:      2934
## Residual Deviance: 2878  AIC: 2884
## log likelihood:  -1439.239
## Nagelkerke R2:  0.02595557
## % pres/err predicted correctly:  -817.2054
## % of predictable range [ (model-null)/(1-null) ]:  0.01332219
## *****
## model index:  3
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##       data = PosDat)
##
## Coefficients:
## (Intercept)          pos      log_freq  pos:log_freq
##      2.713475      -0.151979      0.107958      -0.009398
##
## Degrees of Freedom: 4226 Total (i.e. Null);  4223 Residual
## Null Deviance:      2934
## Residual Deviance: 2878  AIC: 2886
## log likelihood:  -1438.92
## Nagelkerke R2:  0.02625399
## % pres/err predicted correctly:  -817.0748
## % of predictable range [ (model-null)/(1-null) ]:  0.01347974
## *****
## model index:  16
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##       data = PosDat)

```

```

##
## Coefficients:
## (Intercept)          pos
##      2.7238      -0.1555
##
## Degrees of Freedom: 4226 Total (i.e. Null);  4225 Residual
## Null Deviance:      2934
## Residual Deviance: 2885 AIC: 2889
## log likelihood: -1442.56
## Nagelkerke R2:  0.02285416
## % pres/err predicted correctly: -818.4835
## % of predictable range [ (model-null)/(1-null) ]:  0.01178093
## *****
## model index:  19
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##      data = PosDat)
##
## Coefficients:
## (Intercept)      I(pos^2)          pos
##      2.92200      0.01223      -0.26786
##
## Degrees of Freedom: 4226 Total (i.e. Null);  4224 Residual
## Null Deviance:      2934
## Residual Deviance: 2884 AIC: 2890
## log likelihood: -1441.793
## Nagelkerke R2:  0.02357081
## % pres/err predicted correctly: -818.3749
## % of predictable range [ (model-null)/(1-null) ]:  0.0119119
## *****
## model index:  1
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##      data = PosDat)
##
## Coefficients:
##      (Intercept)          stimlen      log_freq  stimlen:log_freq
##      3.16849      -0.13942      0.24610      -0.02406
##
## Degrees of Freedom: 4226 Total (i.e. Null);  4223 Residual
## Null Deviance:      2934
## Residual Deviance: 2900 AIC: 2908
## log likelihood: -1449.876
## Nagelkerke R2:  0.01600443
## % pres/err predicted correctly: -821.848
## % of predictable range [ (model-null)/(1-null) ]:  0.007723692
## *****
## model index:  2
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##      data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen      log_freq

```

```
##      3.20789      -0.14226      0.05255
##
## Degrees of Freedom: 4226 Total (i.e. Null);  4224 Residual
## Null Deviance:      2934
## Residual Deviance: 2902  AIC: 2908
## log likelihood:  -1450.961
## Nagelkerke R2:   0.01498711
## % pres/err predicted correctly:  -822.1516
## % of predictable range [ (model-null)/(1-null) ]:  0.007357583
## *****
## model index:  15
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##      data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen
##      3.3279      -0.1583
##
## Degrees of Freedom: 4226 Total (i.e. Null);  4225 Residual
## Null Deviance:      2934
## Residual Deviance: 2906  AIC: 2910
## log likelihood:  -1452.882
## Nagelkerke R2:   0.01318384
## % pres/err predicted correctly:  -822.811
## % of predictable range [ (model-null)/(1-null) ]:  0.006562388
## *****
## model index:  14
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##      data = PosDat)
##
## Coefficients:
## (Intercept)
##      2.088
##
## Degrees of Freedom: 4226 Total (i.e. Null);  4226 Residual
## Null Deviance:      2934
## Residual Deviance: 2934  AIC: 2936
## log likelihood:  -1466.873
## Nagelkerke R2:   0
## % pres/err predicted correctly:  -828.2529
## % of predictable range [ (model-null)/(1-null) ]:  0
## *****
```

```
BestFLPModel<-FLPRes$ModelResult[[1]]
BestFLPModelFormula<-FLPRes$Model[[1]]

FLPAICSummary<-data.frame(Model=FLPRes$Model,
                           AIC=FLPRes$AIC,row.names=FLPRes$Model)
FLPAICSummary$DeltaAIC<-FLPAICSummary$AIC-FLPAICSummary$AIC[1]
FLPAICSummary$AICexp<-exp(-0.5*FLPAICSummary$DeltaAIC)
FLPAICSummary$AICwt<-FLPAICSummary$AICexp/sum(FLPAICSummary$AICexp)
FLPAICSummary$NagR2<-FLPRes$NagR2
```

```

FLPAICSummary <- merge(FLPAICSummary,FLPRes$CoefficientValues,
                        by='row.names',sort=FALSE)
FLPAICSummary <- subset(FLPAICSummary, select = -c(Row.names))

write.csv(FLPAICSummary,paste0(TablesDir,CurPat,"_",
                               RunLabel,"_",CurTask,
                               "_freq_len_pos_model_summary.csv"),row.names = FALSE)

kable(FLPAICSummary)

```

Model	AIC Delta	AIC	AICw	NagR ²	Intercept	log_stimlen	log_pos	log_freq	I(pos^2)	log_freq*I(pos^2)	len:I(pos^2)
preserved ~ stimlen * (I(pos^2) + pos)	2872.9500	0.000000	0.0000	0.8967	3840.1917	0.1213	0.0422	NA	0.9579	NA	- NA NA - 0.0134346 0.0775600 0.1689052
preserved ~ stimlen + (I(pos^2) + pos) *	2879.6433	0.004041	0.7327	0.9323	4455.520	- NA	- 0.1141	NA	0.0139025	NA	NA NA
log_freq preserved ~ stimlen *	2880.7820	0.003328	0.9243	0.9323	4455.2503	-	-	- NA	0.1130	0.0174	NA NA
log_freq + (I(pos^2) + pos) *	2880.7820	0.003328	0.9243	0.9323	4455.2503	-	-	- NA	0.1130	0.0174	NA NA
log_freq preserved ~ stimlen + I(pos^2) + pos + log_freq	2881.2223	0.011500	0.9362	0.9323	4455.2503	0.0532	NA	- NA	0.0171	NA	NA NA
preserved ~ stimlen + I(pos^2) + pos + log_freq	2881.2223	0.011500	0.9362	0.9323	4455.2503	0.0532	NA	- NA	0.0171	NA	NA NA
preserved ~ stimlen *	2881.8354	0.015785	0.9304	0.9323	4455.2503	0.2378	166	- NA	0.0165	NA	NA NA
log_freq + I(pos^2) + pos	2881.8354	0.015785	0.9304	0.9323	4455.2503	0.2378	166	- NA	0.0165	NA	NA NA
preserved ~ (I(pos^2) + pos) *	2881.8973	0.017331	0.9298	0.9323	4455.2503	0.1096	658	- NA	0.0133	NA	NA NA
log_freq preserved ~ stimlen *	2882.0371	0.021091	0.9290	0.9323	4455.2503	0.2464	951	- NA	0.0097	0.078	NA NA NA
log_freq + pos	2882.0371	0.021091	0.9290	0.9323	4455.2503	0.2464	951	- NA	0.0097	0.078	NA NA NA
preserved ~ stimlen + pos + log_freq	2882.1211	0.009994	0.9362	0.9323	4455.2503	0.0537	NA	- NA	0.0097	0.078	NA NA NA
preserved ~ stimlen + I(pos^2) + pos	2883.1019	0.018260	0.9253	0.9323	4455.2503	0.0730	NA	- NA	0.0173	NA	NA NA
preserved ~ stimlen + pos * log_freq	2883.5640	0.043061	0.8636	0.9323	4455.2503	0.0823	NA	-	0.1290	0.065	NA NA NA

Model	AIC	Delta AIC	AICw	NagR ²	Intercept	stimlen	log_freq	log_pos	log_freq:I(pos)	I(pos^2)	log_freq:I(pos)	stimlen:I(pos^2)
preserved ~ stimlen * pos	2883.883	23.354	0.406	0.387	27207	NA	NA	0.076	NA	NA	NA	NA
					0.0088790							0.0233572
preserved ~ stimlen *	2884.127	67.405	0.301	0.385	28913	0.246	7148	-	NA	0.0008099	NA	NA
					0.0697540	0.0244561	265854					
log_freq + pos *												
log_freq												
preserved ~ stimlen + pos	2884.116	200.606	0.369	0.306	26330	0.0274	NA	NA	-	NA	NA	NA
					0.0890179			0.126	5568			
preserved ~ pos + log_freq	2884.117	519.526	0.301	0.270	25953	0.067	3844	-	NA	NA	NA	NA
								0.148	8902			
preserved ~ pos *	2885.133	799.081	0.665	0.662	25743	0.107	9543	-	-	NA	NA	NA
								0.151	10923	978		
log_freq												
preserved ~ pos	2889.112	160.400	0.095	0.622	28778	NA	NA	-	NA	NA	NA	NA
								0.155	4804			
preserved ~ I(pos^2) + pos	2889.587	273.702	0.400	0.123	30800	NA	NA	-	NA	NA	0.012	2337
								0.267	8608			
preserved ~ stimlen *	2907.343	393.630	0.000	0.001	63168	0.246	1014	NA	NA	NA	NA	NA
						0.139	4159	0.024	0623			
log_freq												
preserved ~ stimlen + log_freq	2907.322	623.000	0.000	0.001	4927	0.052	5471	NA	NA	NA	NA	NA
								0.142	2621			
preserved ~ stimlen	2909.363	304.230	0.000	0.001	3328	0.246	726	NA	NA	NA	NA	NA
								0.158	3129			
preserved ~ 1	2935.627	585.800	0.000	0.000	2038	NA	NA	NA	NA	NA	NA	NA

```
print(BestFLPModelFormula)
```

```
## [1] "preserved ~ stimlen * (I(pos^2) + pos)"
```

```
print(BestFLPModel)
```

```
##
```

```
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
```

```
## data = PosDat)
```

```
##
```

```
## Coefficients:
```

```
## (Intercept)          stimlen          I(pos^2)          pos  stimlen:I(pos^2)          stimlen:I(pos^2)
```

```
## 0.91712          0.27304          -0.07756          0.95798          0.01343          -0
```

```
##
```

```
## Degrees of Freedom: 4226 Total (i.e. Null); 4221 Residual
```

```
## Null Deviance: 2934
```

```
## Residual Deviance: 2861 AIC: 2873
```

```
# do a median split on frequency to plot hf/lf effects (analysis is continuous)
```

```
median_freq <- median(PosDat$log_freq)
```

```
PosDat$freq_bin[PosDat$log_freq >= median_freq] <- "hf"
```

```
PosDat$freq_bin[PosDat$log_freq < median_freq] <- "lf"
```

```

PosDat$FLPFitted<-fitted(BestFLPModel)

HFDat <- PosDat[PosDat$freq_bin == "hf",]
LFDat <- PosDat[PosDat$freq_bin == "lf",]

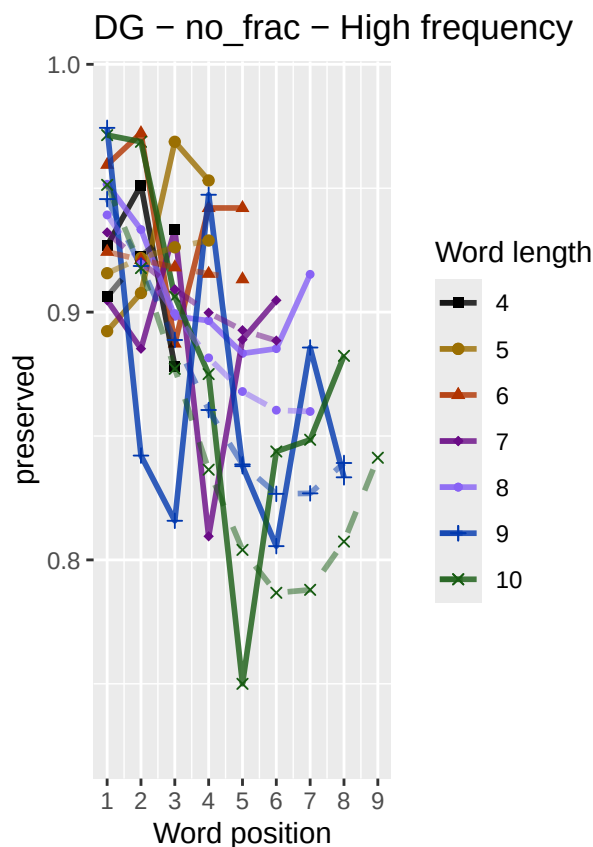
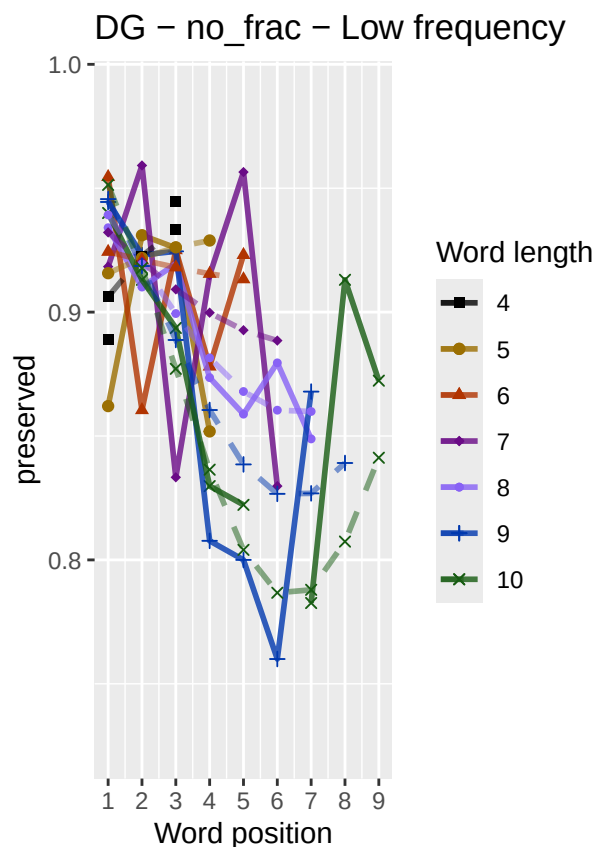
HF_Plot <- plot_len_pos_obs_predicted(HFDat,paste0(CurPat," - ",RunLabel," - High frequency"),"FLPFitted")

## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## Scale for y is already present. Adding another scale for y, which will replace the existing scale.
LF_Plot <- plot_len_pos_obs_predicted(LFDat,paste0(CurPat," - ",RunLabel," - Low frequency"),"FLPFitted")

## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## Scale for y is already present. Adding another scale for y, which will replace the existing scale.
library(ggpubr)
Both_Plots <- ggarrange(LF_Plot,HF_Plot) # labels=c("LF","HF",ncol=2)

## Warning: Removed 3 rows containing missing values or values outside the scale range (`geom_point()`).
## Warning: Removed 1 row containing missing values or values outside the scale range (`geom_line()`).
## Warning: Removed 1 row containing missing values or values outside the scale range (`geom_point()`).
## Warning: Removed 1 row containing missing values or values outside the scale range (`geom_line()`).
ggsave(paste0(FigDir,CurPat,"_",RunLabel,"_",
              CurTask,"_frequency_effect_length_pos_wfit.png"),
       device="png",unit="cm",width=30,height=11)
print(Both_Plots)

```



```
# only main effects
MEModelEquations<-c(
  "preserved ~ CumPres",
  "preserved ~ CumErr",
  "preserved ~ (I(pos^2)+pos)",
  "preserved ~ pos",
  "preserved ~ stimlen",
  "preserved ~ 1"
)
MERes<-TestModels(MEModelEquations,PosDat)
```

```
## *****
## model index: 2
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      CumErr
##      2.3481      -0.7024
##
## Degrees of Freedom: 4226 Total (i.e. Null);  4225 Residual
## Null Deviance:      2934
## Residual Deviance: 2818 AIC: 2822
## log likelihood:  -1409.232
## Nagelkerke R2:  0.05375928
## % pres/err predicted correctly:  -799.5156
```

```

## % of predictable range [ (model-null)/(1-null) ]: 0.03465446
## *****
## model index: 4
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)          pos
##      2.7238      -0.1555
##
## Degrees of Freedom: 4226 Total (i.e. Null); 4225 Residual
## Null Deviance:      2934
## Residual Deviance: 2885 AIC: 2889
## log likelihood: -1442.56
## Nagelkerke R2: 0.02285416
## % pres/err predicted correctly: -818.4835
## % of predictable range [ (model-null)/(1-null) ]: 0.01178093
## *****
## model index: 3
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      I(pos^2)          pos
##      2.92200      0.01223      -0.26786
##
## Degrees of Freedom: 4226 Total (i.e. Null); 4224 Residual
## Null Deviance:      2934
## Residual Deviance: 2884 AIC: 2890
## log likelihood: -1441.793
## Nagelkerke R2: 0.02357081
## % pres/err predicted correctly: -818.3749
## % of predictable range [ (model-null)/(1-null) ]: 0.0119119
## *****
## model index: 5
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen
##      3.3279      -0.1583
##
## Degrees of Freedom: 4226 Total (i.e. Null); 4225 Residual
## Null Deviance:      2934
## Residual Deviance: 2906 AIC: 2910
## log likelihood: -1452.882
## Nagelkerke R2: 0.01318384
## % pres/err predicted correctly: -822.811
## % of predictable range [ (model-null)/(1-null) ]: 0.006562388
## *****
## model index: 1

```



```

##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept) CumPres
## 2.22196 -0.05232
##
## Degrees of Freedom: 4226 Total (i.e. Null); 4225 Residual
## Null Deviance: 2934
## Residual Deviance: 2929 AIC: 2933
## log likelihood: -1464.686
## Nagelkerke R2: 0.002066272
## % pres/err predicted correctly: -827.4197
## % of predictable range [ (model-null)/(1-null) ]: 0.001004801
## *****
## model index: 6
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)
## 2.088
##
## Degrees of Freedom: 4226 Total (i.e. Null); 4226 Residual
## Null Deviance: 2934
## Residual Deviance: 2934 AIC: 2936
## log likelihood: -1466.873
## Nagelkerke R2: 0
## % pres/err predicted correctly: -828.2529
## % of predictable range [ (model-null)/(1-null) ]: 0
## *****

BestMEModel<-MERes$ModelResult[[ BestModelIndexL1 ]]
BestMEModelFormula<-MERes$Model[[BestModelIndexL1]]

MEAICSummary<-data.frame(Model=MERes$Model,
                          AIC=MERes$AIC,row.names=MERes$Model)
MEAICSummary$DeltaAIC<-MEAICSummary$AIC-MEAICSummary$AIC[1]
MEAICSummary$AICexp<-exp(-0.5*MEAICSummary$DeltaAIC)
MEAICSummary$AICwt<-MEAICSummary$AICexp/sum(MEAICSummary$AICexp)
MEAICSummary$NagR2<-MERes$NagR2

MEAICSummary <- merge(MEAICSummary,MERes$CoefficientValues,
                      by='row.names',sort=FALSE)
MEAICSummary <- subset(MEAICSummary, select = -c(Row.names))

write.csv(MEAICSummary,paste0(TablesDir,CurPat,"_",RunLabel,"_",
                             CurTask,"_main_effects_model_summary.csv"),
          row.names = FALSE)
kable(MEAICSummary)

```

Model	AIC	DeltaAIC	AICexp	AICwt	NagR2	(Intercept)	CumPres	CumErr	I(pos^2)	pos	stimlen
preserved ~ CumErr	2822.463	0.00000	1	1	0.053759	2.348075	NA	-	NA	NA	NA
preserved ~ pos	2889.126	66.65605	0	0	0.022854	2.723778	NA	NA	NA	-	NA
preserved ~ (I(pos^2) + pos)	2889.587	67.12222	0	0	0.023570	2.922003	NA	NA	0.0122337	-	NA
preserved ~ stimlen	2909.763	87.29888	0	0	0.013183	3.327873	NA	NA	NA	NA	-
preserved ~ CumPres	2933.372	110.90719	0	0	0.002066	3.221961	-	NA	NA	NA	NA
preserved ~ 1	2935.745	13.28045	0	0	0.000000	0.088255	0.0523184	NA	NA	NA	NA

```

if(DoSimulations){
  BestMEModelFormulaRnd <- BestMEModelFormula
  if(grepl("CumPres",BestMEModelFormulaRnd)){
    BestMEModelFormulaRnd <- gsub("CumPres","RndCumPres",BestMEModelFormulaRnd)
  }else if(grepl("CumErr",BestMEModelFormulaRnd)){
    BestMEModelFormulaRnd <- gsub("CumErr","RndCumErr",BestMEModelFormulaRnd)
  }

  RndModelAIC<-numeric(length=RandomSamples)
  for(rindex in seq(1,RandomSamples)){
    # Shuffle cumulative values
    PosDat$RndCumPres <- ShuffleWithinWord(PosDat,"CumPres")
    PosDat$RndCumErr <- ShuffleWithinWord(PosDat,"CumErr")
    BestModelRnd <- glm(as.formula(BestMEModelFormulaRnd),
                        family="binomial",data=PosDat)
    RndModelAIC[rindex] <- BestModelRnd$aic
  }
  ModelNames<-c(paste0("***",BestMEModelFormula),
                 rep(BestMEModelFormulaRnd,RandomSamples))
  AICValues <- c(BestMEModel$aic,RndModelAIC)
  BestMEModelRndDF <- data.frame(Name=ModelNames,AIC=AICValues)
  BestMEModelRndDF <- BestMEModelRndDF %>% arrange(AIC)
  BestMEModelRndDF <- rbind(BestMEModelRndDF,
                            data.frame(Name=c("Random average"),
                                         AIC=c(mean(RndModelAIC))))
  BestMEModelRndDF <- rbind(BestMEModelRndDF,
                            data.frame(Name=c("Random SD"),
                                         AIC=c(sd(RndModelAIC))))

  write.csv(BestMEModelRndDF,
            paste0(TablesDir,CurPat,"_",RunLabel,"_",CurTask,
                  "_best_main_effects_model_with_random_cum_term.csv"),
            row.names = FALSE)
}

```

```

syll_component_summary <- PosDat %>%
  group_by(syll_component) %>% summarise(MeanPres = mean(preserved),
                                         N = n())
write.csv(syll_component_summary,paste0(TablesDir,CurPat,"_",
                                       RunLabel,"_",CurTask,

```

```

kable(syll_component_summary)                                     "_syllable_component_summary.csv"),row.names = FALSE)

```

syll_component	MeanPres	N
1	0.9067642	547
O	0.8669725	1962
P	1.0000000	36
S	0.7883817	241
V	0.9285219	1441

```

# main effects models for data without satellite positions

```

```

keep_components = c("0","V","1")
OV1Data <- PosDat[PosDat$syll_component %in% keep_components,]
OV1Data <- OV1Data %>% select(stim_number,
                             stimlen,stim,pos,
                             preserved,syll_component)
OV1Data$CumPres <- CalcCumPres(OV1Data)
OV1Data$CumErr <- CalcCumErrFromPreserved(OV1Data)

SimpSyllMEAICSummary <- EvaluateSubsetData(OV1Data,MEModelEquations)

```

```

## *****
## model index: 2
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      CumErr
##      2.3705      -0.7052
##
## Degrees of Freedom: 3949 Total (i.e. Null); 3948 Residual
## Null Deviance:      2655
## Residual Deviance: 2563 AIC: 2567
## log likelihood: -1281.395
## Nagelkerke R2: 0.04711606
## % pres/err predicted correctly: -720.3015
## % of predictable range [ (model-null)/(1-null) ]: 0.0289392
## *****
## model index: 4
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      pos
##      2.7683      -0.1523
##
## Degrees of Freedom: 3949 Total (i.e. Null); 3948 Residual
## Null Deviance:      2655
## Residual Deviance: 2613 AIC: 2617

```

```

## log likelihood: -1306.562
## Nagelkerke R2: 0.02151613
## % pres/err predicted correctly: -733.7615
## % of predictable range [ (model-null)/(1-null) ]: 0.0108185
## *****
## model index: 3
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept) I(pos^2) pos
## 2.95855 0.01172 -0.25996
##
## Degrees of Freedom: 3949 Total (i.e. Null); 3947 Residual
## Null Deviance: 2655
## Residual Deviance: 2612 AIC: 2618
## log likelihood: -1305.929
## Nagelkerke R2: 0.02216375
## % pres/err predicted correctly: -733.6422
## % of predictable range [ (model-null)/(1-null) ]: 0.01097917
## *****
## model index: 5
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept) stimlen
## 3.205 -0.136
##
## Degrees of Freedom: 3949 Total (i.e. Null); 3948 Residual
## Null Deviance: 2655
## Residual Deviance: 2636 AIC: 2640
## log likelihood: -1318.104
## Nagelkerke R2: 0.009665432
## % pres/err predicted correctly: -738.2744
## % of predictable range [ (model-null)/(1-null) ]: 0.004742998
## *****
## model index: 1
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept) CumPres
## 2.25569 -0.04727
##
## Degrees of Freedom: 3949 Total (i.e. Null); 3948 Residual
## Null Deviance: 2655
## Residual Deviance: 2652 AIC: 2656
## log likelihood: -1325.967
## Nagelkerke R2: 0.001551971
## % pres/err predicted correctly: -741.2415

```

```
## % of predictable range [ (model-null)/(1-null) ]: 0.0007485071
## *****
## model index: 6
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)
## 2.142
##
## Degrees of Freedom: 3949 Total (i.e. Null); 3949 Residual
## Null Deviance: 2655
## Residual Deviance: 2655 AIC: 2657
## log likelihood: -1327.468
## Nagelkerke R2: -4.537235e-16
## % pres/err predicted correctly: -741.7975
## % of predictable range [ (model-null)/(1-null) ]: 0
## *****
```

```
write.csv(SimpSyllMEAICSummary,
  paste0(TablesDir, CurPat, "_", RunLabel, "_",
    CurTask,
    "_OV1_main_effects_model_summary.csv"),
  row.names = FALSE)
kable(SimpSyllMEAICSummary)
```

Model	AIC	DeltaAIC	AICexp	AICwt	NagR2	(Intercept)	CumPres	CumErr	I(pos^2)	pos	stimlen
preserved ~ CumErr	2566.791	0.00000	1	1	0.0471162	1.370526	NA	-	NA	NA	NA
preserved ~ pos	2617.123	50.33222	0	0	0.0215162	1.768292	NA	NA	NA	-	NA
										0.1522677	
preserved ~ (I(pos^2) + pos)	2617.858	51.06680	0	0	0.0221632	1.958546	NA	NA	0.0117158	-	NA
										0.2599567	
preserved ~ stimlen	2640.208	73.41656	0	0	0.0096653	1.205121	NA	NA	NA	NA	-
											0.1360122
preserved ~ CumPres	2655.933	89.14358	0	0	0.0015520	0.255690	-	NA	NA	NA	NA
							0.04727				
preserved ~ 1	2656.936	90.14478	0	0	0.0000000	0.142190	NA	NA	NA	NA	NA

```
# main effects models for data without satellite or coda positions
# (tradeoff -- takes away more complex segments, in addition to satellites, but
# also reduces data)
```

```
keep_components = c("0", "V")
OVData <- PosDat[PosDat$syll_component %in% keep_components,]
OVData <- OVData %>% select(stim_number,
  stimlen, stim, pos,
  preserved, syll_component)
OVData$CumPres <- CalcCumPres(OVData)
OVData$CumErr <- CalcCumErrFromPreserved(OVData)

SimpSyllMEAICSummary2 <- EvaluateSubsetData(OVData, MEModelEquations)
```

```

## *****
## model index: 2
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept) CumErr
## 2.3056 -0.7021
##
## Degrees of Freedom: 3402 Total (i.e. Null); 3401 Residual
## Null Deviance: 2315
## Residual Deviance: 2260 AIC: 2264
## log likelihood: -1130.15
## Nagelkerke R2: 0.0322321
## % pres/err predicted correctly: -636.2643
## % of predictable range [ (model-null)/(1-null) ]: 0.0197893
## *****
## model index: 4
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept) pos
## 2.7438 -0.1515
##
## Degrees of Freedom: 3402 Total (i.e. Null); 3401 Residual
## Null Deviance: 2315
## Residual Deviance: 2276 AIC: 2280
## log likelihood: -1138.242
## Nagelkerke R2: 0.02272652
## % pres/err predicted correctly: -641.6668
## % of predictable range [ (model-null)/(1-null) ]: 0.01147942
## *****
## model index: 3
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept) I(pos^2) pos
## 2.94845 0.01299 -0.27021
##
## Degrees of Freedom: 3402 Total (i.e. Null); 3400 Residual
## Null Deviance: 2315
## Residual Deviance: 2275 AIC: 2281
## log likelihood: -1137.507
## Nagelkerke R2: 0.02359077
## % pres/err predicted correctly: -641.5179
## % of predictable range [ (model-null)/(1-null) ]: 0.01170841
## *****
## model index: 5
##

```

```

## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen
##      3.0497      -0.1192
##
## Degrees of Freedom: 3402 Total (i.e. Null); 3401 Residual
## Null Deviance:      2315
## Residual Deviance: 2302 AIC: 2306
## log likelihood: -1150.994
## Nagelkerke R2: 0.007653595
## % pres/err predicted correctly: -646.6468
## % of predictable range [ (model-null)/(1-null) ]: 0.003819349
## *****
## model index: 1
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      CumPres
##      2.27758      -0.07634
##
## Degrees of Freedom: 3402 Total (i.e. Null); 3401 Residual
## Null Deviance:      2315
## Residual Deviance: 2310 AIC: 2314
## log likelihood: -1154.795
## Nagelkerke R2: 0.003139241
## % pres/err predicted correctly: -648.1181
## % of predictable range [ (model-null)/(1-null) ]: 0.00155627
## *****
## model index: 6
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)
##      2.122
##
## Degrees of Freedom: 3402 Total (i.e. Null); 3402 Residual
## Null Deviance:      2315
## Residual Deviance: 2315 AIC: 2317
## log likelihood: -1157.433
## Nagelkerke R2: -4.49933e-16
## % pres/err predicted correctly: -649.1299
## % of predictable range [ (model-null)/(1-null) ]: 0
## *****
write.csv(SimpSyllMEAICSummary2,
         paste0(TablesDir, CurPat, "_", RunLabel, "_", CurTask,
               "_OV_main_effects_model_summary.csv"), row.names = FALSE)
kable(SimpSyllMEAICSummary2)

```

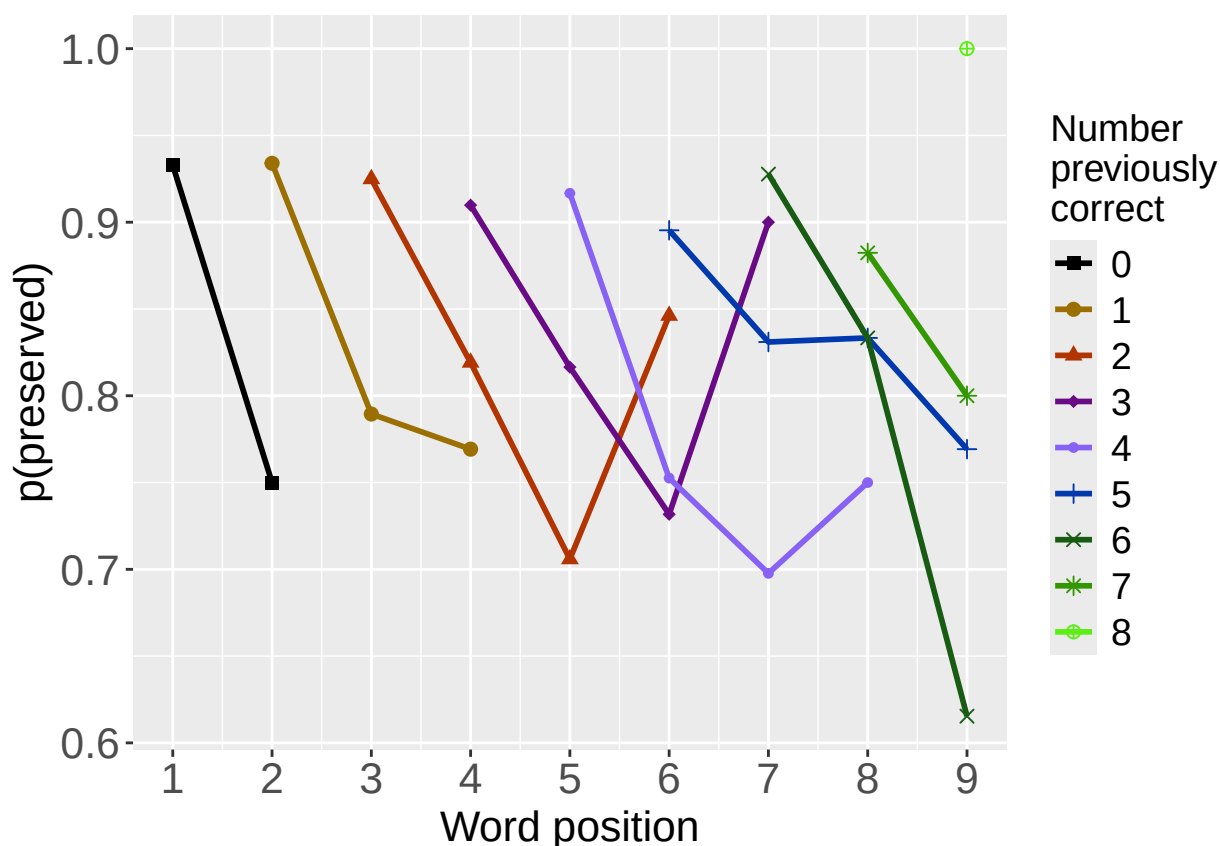
Model	AIC	DeltaAIC	AICexp	AICwt	NagR2	(Intercept)	CumPres	CumErr	I(pos^2)	pos	stimlen
preserved ~ CumErr	2264.300	0.00000	1.0000000	0.9994596	0.0322321	3.305607	NA	-	NA	NA	NA
preserved ~ pos	2280.481	16.18316	0.0003061	0.0003059	0.0227263	3.743796	NA	NA	NA	-	NA
								0.7020899		0.1514599	
preserved ~ (I(pos^2) + pos)	2281.011	16.71497	0.0002346	0.0002345	0.0235902	3.948451	NA	NA	0.012989	-	NA
									0.2702061		
preserved ~ stimlen	2305.981	41.68796	0.0000000	0.0000000	0.0076533	3.049661	NA	NA	NA	NA	-
											0.119232
preserved ~ CumPres	2313.590	19.28962	0.0000000	0.0000000	0.0031392	2.277578	-	NA	NA	NA	NA
							0.0763424				
preserved ~ 1	2316.865	2.56575	0.0000000	0.0000000	0.0000000	2.122130	NA	NA	NA	NA	NA

```
# plot prev err and prev cor plots
```

```
PrevCorPlot<-PlotObsPreviousCorrect(PosDat,palette_values,shape_values)
```

```
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
```

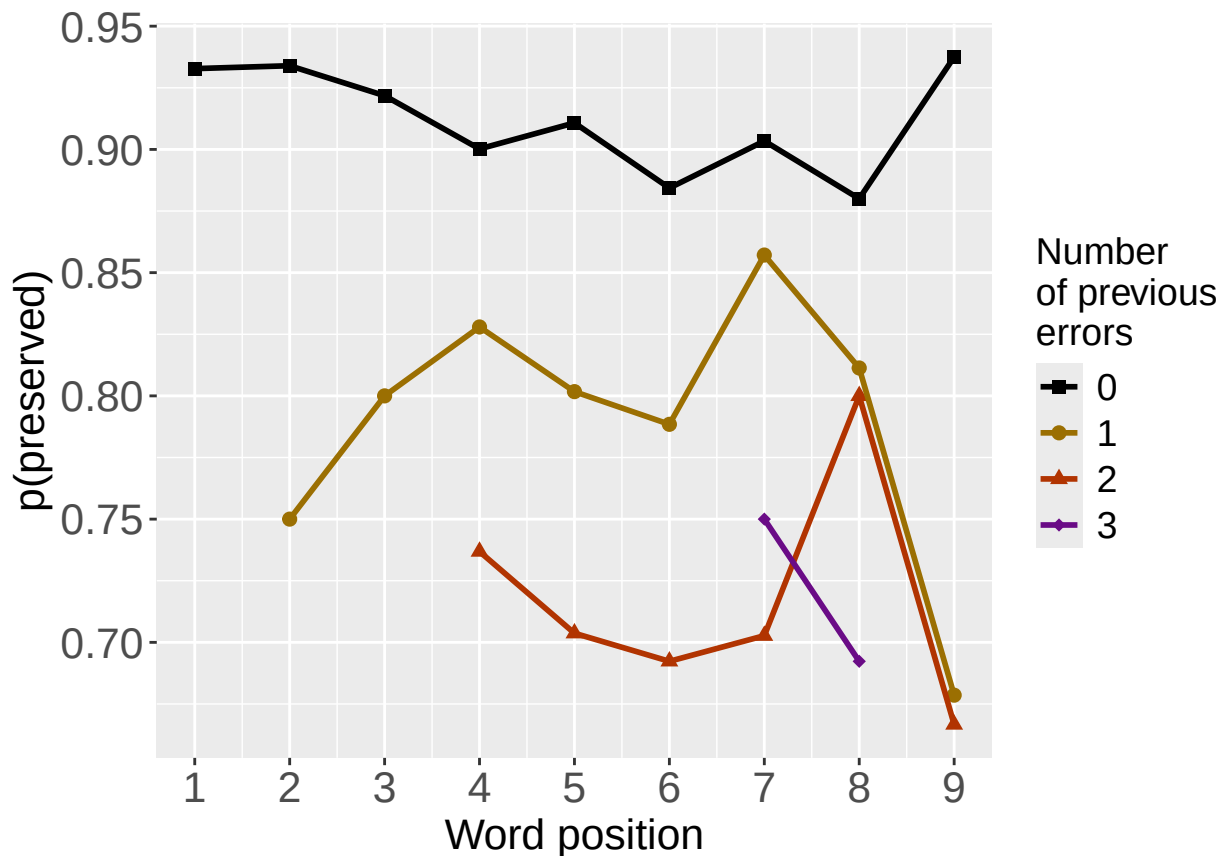
```
print(PrevCorPlot)
```



```
PrevErrPlot<-PlotObsPreviousError(PosDat,palette_values,shape_values)
```

```
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
```

```
print(PrevErrPlot)
```

```

PlotName<-paste0(FigDir,CurPat,"_",RunLabel,"_",CurTask,"_PrevCorPlot_Obs")
ggsave(filename=paste0(PlotName,".tif"),plot=PrevCorPlot,device="tiff",compression = "lzw")

## Saving 6.5 x 4.5 in image

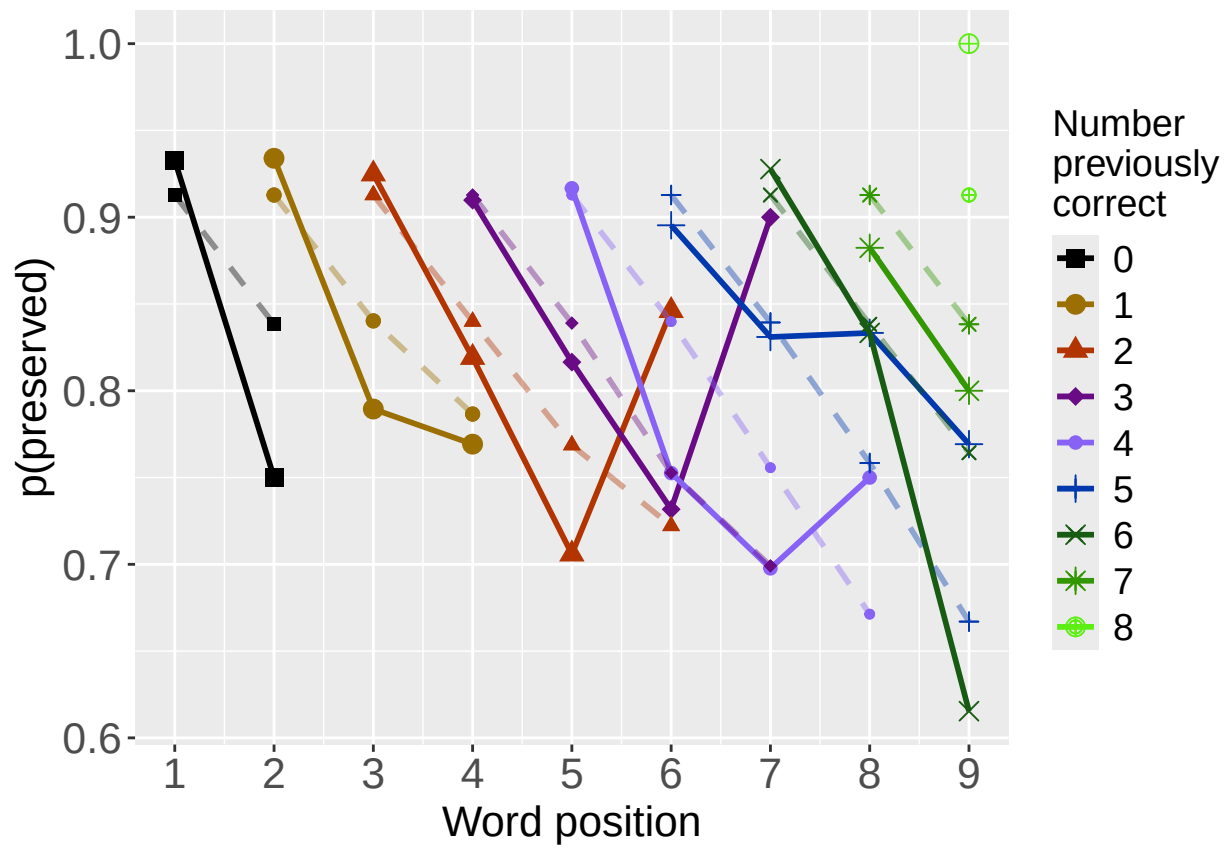
PlotName<-paste0(FigDir,CurPat,"_",RunLabel,"_",CurTask,"_PrevErrPlot_Obs")
ggsave(filename=paste0(PlotName,".tif"),plot=PrevErrPlot,device="tiff",compression = "lzw")

## Saving 6.5 x 4.5 in image
# plot prev err and prev cor with predicted values
MEModel<-MERes$ModelResult[[ BestModelIndexL1 ]]
PosDat$MEPred<-fitted(MEModel)

PrevCorPlot<-PlotObsPredPreviousCorrect(PosDat,"preserved","MEPred",palette_values,shape_values)

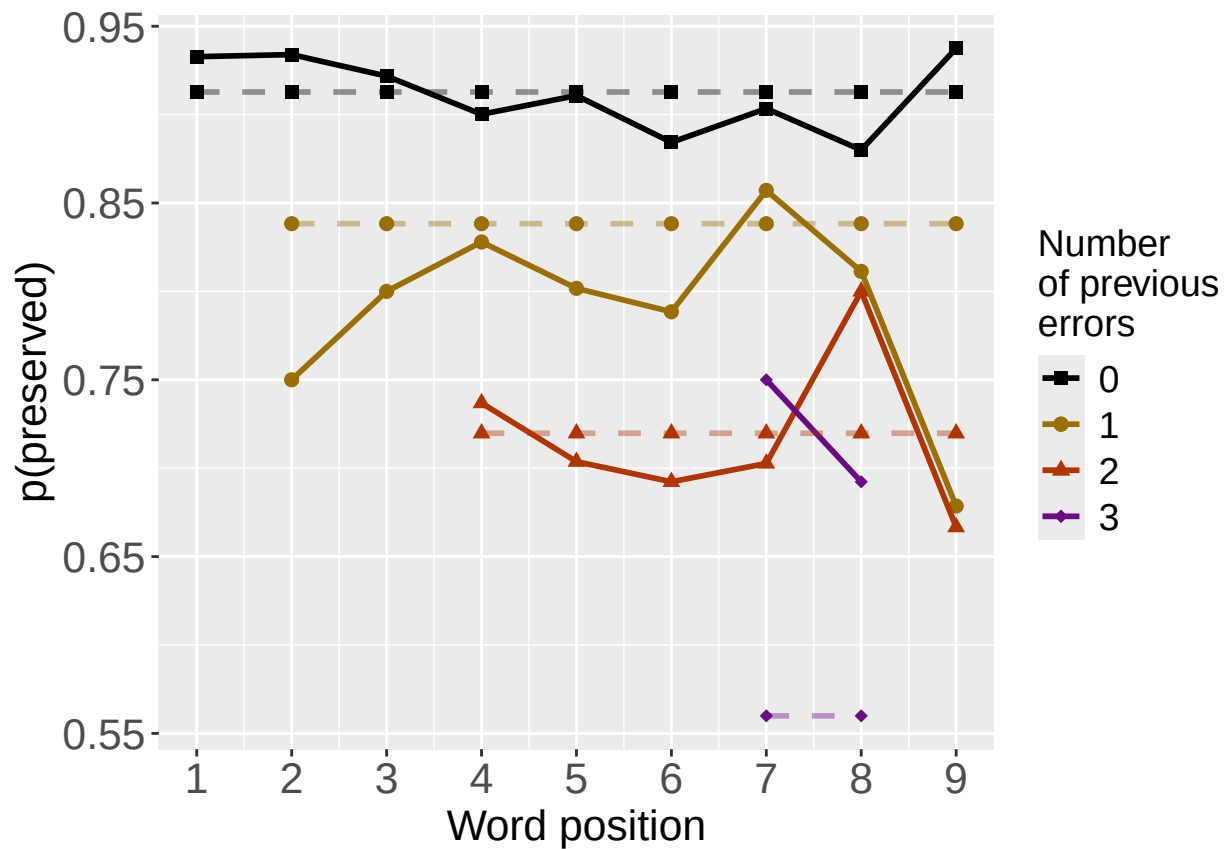
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
print(PrevCorPlot)

```



```
PrevErrPlot<-PlotObsPredPreviousError(PosDat,"preserved","MEPred",palette_values,shape_values)

## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
print(PrevErrPlot)
```



```

PlotName<-paste0(FigDir,CurPat,"_",RunLabel,"_",
                  CurTask,"PrevCorPlot_ObsPredME")
ggsave(filename=paste0(PlotName,".tif"),plot=PrevCorPlot,device="tiff",compression = "lzw")

## Saving 6.5 x 4.5 in image

PlotName<-paste0(FigDir,CurPat,"_",RunLabel,"_",
                  CurTask,"PrevErrPlot_ObsPredME")
ggsave(filename=paste0(PlotName,".tif"),plot=PrevErrPlot,device="tiff",compression = "lzw")

## Saving 6.5 x 4.5 in image

```

```

CumAICSummary <- NULL

#####
# level 2 -- Add position squared (quadratic with position)
#####

# After establishing the primary variable, see about additions

BestMEFactor<-BestMEModelFormula
OtherFactor<-"I(pos^2) + pos"
OtherFactorName<-"quadratic_by_pos"

if(!grepl("pos",BestMEFactor,fixed = TRUE)){ # if best model does not include pos
  AICSummary<-TestLevel2Models(PosDat,BestMEFactor,OtherFactor,OtherFactorName)
  CumAICSummary <- AICSummary
  kable(AICSummary)
}

```

```

## *****
## model index: 2
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##        data = PosDat)
##
## Coefficients:
## (Intercept)      CumErr      I(pos^2)         pos
##      2.90128      -0.63063      0.02109      -0.25227
##
## Degrees of Freedom: 4226 Total (i.e. Null);  4223 Residual
## Null Deviance:      2934
## Residual Deviance: 2808  AIC: 2816
## log likelihood:  -1404.101
## Nagelkerke R2:  0.05847458
## % pres/err predicted correctly:  -797.9023
## % of predictable range [ (model-null)/(1-null) ]:  0.03659993
## *****
## model index: 1
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",

```

```

##      data = PosDat)
##
## Coefficients:
## (Intercept)      CumErr
##      2.3481      -0.7024
##
## Degrees of Freedom: 4226 Total (i.e. Null);  4225 Residual
## Null Deviance:      2934
## Residual Deviance: 2818  AIC: 2822
## log likelihood:  -1409.232
## Nagelkerke R2:  0.05375928
## % pres/err predicted correctly:  -799.5156
## % of predictable range [ (model-null)/(1-null) ]:  0.03465446
## *****
## model index:  3
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##      data = PosDat)
##
## Coefficients:
## (Intercept)      I(pos^2)          pos
##      2.92200      0.01223      -0.26786
##
## Degrees of Freedom: 4226 Total (i.e. Null);  4224 Residual
## Null Deviance:      2934
## Residual Deviance: 2884  AIC: 2890
## log likelihood:  -1441.793
## Nagelkerke R2:  0.02357081
## % pres/err predicted correctly:  -818.3749
## % of predictable range [ (model-null)/(1-null) ]:  0.0119119
## *****

```

Model	AIC	DeltaAIC	AICexp	AICwt	NagR2	(Intercept)	CumErr	I(pos^2)	pos
preserved ~ CumErr + I(pos^2) + pos	2816.202	0.000000	1.0000000	0.9581732	0.0584746	2.901280	-0.6306296	0.0210856	-0.2522667
preserved ~ CumErr	2822.465	6.262984	0.0436526	0.0418268	0.0537593	2.348075	-0.7024110	NA	NA
preserved ~ I(pos^2) + pos	2889.587	73.385208	0.0000000	0.0000000	0.0235708	2.922003	NA	0.0122337	-0.2678608

```
#####
# level 2 -- Add length
#####

BestMEFactor<-BestMEModelFormula
OtherFactor<-"stimlen"

if(!grepl(OtherFactor,BestMEFactor,fixed = TRUE)){
  AICSummary<-TestLevel2Models(PosDat,BestMEFactor,OtherFactor)
  CumAICSummary <- ConcatAndAddMissingColumns(CumAICSummary,AICSummary)
  kable(AICSummary)
}

## *****
## model index: 2
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept) CumErr stimlen
## 3.1238 -0.6568 -0.1013
##
## Degrees of Freedom: 4226 Total (i.e. Null); 4224 Residual
## Null Deviance: 2934
## Residual Deviance: 2808 AIC: 2814
## log likelihood: -1403.934
## Nagelkerke R2: 0.058628
## % pres/err predicted correctly: -798.1602
## % of predictable range [ (model-null)/(1-null) ]: 0.03628898
## *****
## model index: 1
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept) CumErr
## 2.3481 -0.7024
##
## Degrees of Freedom: 4226 Total (i.e. Null); 4225 Residual
## Null Deviance: 2934
## Residual Deviance: 2818 AIC: 2822
## log likelihood: -1409.232
## Nagelkerke R2: 0.05375928
## % pres/err predicted correctly: -799.5156
## % of predictable range [ (model-null)/(1-null) ]: 0.03465446
## *****
## model index: 3
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
```

```
## (Intercept)      stimlen
##      3.3279      -0.1583
##
## Degrees of Freedom: 4226 Total (i.e. Null);  4225 Residual
## Null Deviance:      2934
## Residual Deviance: 2906  AIC: 2910
## log likelihood:  -1452.882
## Nagelkerke R2:   0.01318384
## % pres/err predicted correctly:  -822.811
## % of predictable range [ (model-null)/(1-null) ]:  0.006562388
## *****
```

Model	AIC	DeltaAIC	AICexp	AICwt	NagR2	(Intercept)	CumErr	stimlen
preserved ~ CumErr + stimlen	2813.867	0.000000	1.0000000	0.9865954	0.0586280	3.123846	- 0.6568245	- 0.1013288
preserved ~ CumErr	2822.465	8.597331	0.0135867	0.0134046	0.0537593	2.348075	- 0.7024110	NA
preserved ~ stimlen	2909.763	95.896214	0.0000000	0.0000000	0.0131838	3.327873	NA	- 0.1583129

```
#####
# level 2 -- add cumulative preserved
#####

BestMEFactor<-BestMEModelFormula
OtherFactor<-"CumPres"

if(!grepl(OtherFactor,BestMEFactor,fixed = TRUE)){
  AICSummary<-TestLevel2Models(PosDat,BestMEFactor,OtherFactor)
  CumAICSummary <- ConcatAndAddMissingColumns(CumAICSummary,AICSummary)
  kable(AICSummary)
}
```

```
## *****
## model index:  1
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##      data = PosDat)
##
## Coefficients:
## (Intercept)      CumErr
##      2.3481      -0.7024
##
## Degrees of Freedom: 4226 Total (i.e. Null);  4225 Residual
## Null Deviance:      2934
## Residual Deviance: 2818  AIC: 2822
## log likelihood:  -1409.232
## Nagelkerke R2:   0.05375928
## % pres/err predicted correctly:  -799.5156
## % of predictable range [ (model-null)/(1-null) ]:  0.03465446
## *****
## model index:  2
##
```

```
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##      data = PosDat)
##
## Coefficients:
## (Intercept)      CumErr      CumPres
##      2.4032      -0.6955      -0.0225
##
## Degrees of Freedom: 4226 Total (i.e. Null);  4224 Residual
## Null Deviance:      2934
## Residual Deviance: 2818  AIC: 2824
## log likelihood:  -1408.863
## Nagelkerke R2:  0.05409877
## % pres/err predicted correctly:  -799.5932
## % of predictable range [ (model-null)/(1-null) ]:  0.03456083
## *****
## model index:  3
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##      data = PosDat)
##
## Coefficients:
## (Intercept)      CumPres
##      2.22196      -0.05232
##
## Degrees of Freedom: 4226 Total (i.e. Null);  4225 Residual
## Null Deviance:      2934
## Residual Deviance: 2929  AIC: 2933
## log likelihood:  -1464.686
## Nagelkerke R2:  0.002066272
## % pres/err predicted correctly:  -827.4197
## % of predictable range [ (model-null)/(1-null) ]:  0.001004801
## *****
```

Model	AIC	DeltaAIC	AICexp	AICwt	NagR2	(Intercept)	CumErr	CumPres
preserved ~ CumErr	2822.465	0.000000	1.0000000	0.6527075	0.0537593	2.348075	-	NA
							0.7024110	
preserved ~ CumErr	2823.726	1.261923	0.5320799	0.3472925	0.0540988	2.403190	-	-
+ CumPres							0.6954848	0.0225008
preserved ~ CumPres	2933.372	110.907188	0.0000000	0.0000000	0.0020663	2.221961	NA	-
								0.0523184

```
#####
# level 2 -- Add linear position (NOT quadratic)
#####

BestMEFactor<-BestMEModelFormula
OtherFactor<-"pos"

if(!grepl(OtherFactor,BestMEFactor,fixed = TRUE)){
  AICSummary<-TestLevel2Models(PosDat,BestMEFactor,OtherFactor)
  CumAICSummary <- ConcatAndAddMissingColumns(CumAICSummary,AICSummary)
  kable(AICSummary)
}
```



```

## *****
## model index: 2
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      CumErr      pos
##      2.5701      -0.6164      -0.0624
##
## Degrees of Freedom: 4226 Total (i.e. Null); 4224 Residual
## Null Deviance:      2934
## Residual Deviance: 2813 AIC: 2819
## log likelihood: -1406.275
## Nagelkerke R2: 0.05647809
## % pres/err predicted correctly: -798.85
## % of predictable range [ (model-null)/(1-null) ]: 0.0354571
## *****
## model index: 1
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      CumErr
##      2.3481      -0.7024
##
## Degrees of Freedom: 4226 Total (i.e. Null); 4225 Residual
## Null Deviance:      2934
## Residual Deviance: 2818 AIC: 2822
## log likelihood: -1409.232
## Nagelkerke R2: 0.05375928
## % pres/err predicted correctly: -799.5156
## % of predictable range [ (model-null)/(1-null) ]: 0.03465446
## *****
## model index: 3
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      pos
##      2.7238      -0.1555
##
## Degrees of Freedom: 4226 Total (i.e. Null); 4225 Residual
## Null Deviance:      2934
## Residual Deviance: 2885 AIC: 2889
## log likelihood: -1442.56
## Nagelkerke R2: 0.02285416
## % pres/err predicted correctly: -818.4835
## % of predictable range [ (model-null)/(1-null) ]: 0.01178093
## *****

```

Model	AIC	DeltaAIC	AICexp	AICwt	NagR2	(Intercept)	CumErr	pos
preserved ~ CumErr + pos	2818.550	0.000000	1.000000	0.8762369	0.0564781	2.570138	- 0.6163625	- 0.0623954
preserved ~ CumErr	2822.465	3.914534	0.1412439	0.1237631	0.0537593	2.348075	- 0.7024110	NA
preserved ~ pos	2889.121	70.570588	0.000000	0.000000	0.0228542	2.723778	NA	- 0.1554804

```
CumAICSummary <- CumAICSummary %>% arrange(AIC)
BestModelFormulaL2 <- CumAICSummary$Model[BestModelIndexL2]
write.csv(CumAICSummary, paste0(TablesDir, CurPat, "_",
                                RunLabel, "_", CurTask,
                                "_main_effects_plus_one_model_summary.csv"), row.names = FALSE)
kable(CumAICSummary)
```

Model	AIC	DeltaAIC	AICexp	AICwt	NagR2	(Intercept)	CumErr	I(pos^2)	pos	stimlen	CumPres
preserved ~ CumErr + stimlen	2813.867	0.000000	1.000000	0.986595	0.1058628	0.123846	- 0.6568245	NA	NA	- 0.1013288	NA
preserved ~ CumErr + I(pos^2) + pos	2816.202	0.000000	1.000000	0.958173	0.2058474	0.6901280	- 0.6306296	0.0210856	- 0.2522667	NA	NA
preserved ~ CumErr + pos	2818.550	0.000000	1.000000	0.8762369	0.0564781	2.570138	- 0.6163625	NA	- 0.0623954	NA	NA
preserved ~ CumErr	2822.465	3.262984	0.043652	0.6041826	0.0537593	2.348075	- 0.7024110	NA	NA	NA	NA
preserved ~ CumErr	2822.465	5.597331	0.013586	0.1013404	0.0537593	2.348075	- 0.7024110	NA	NA	NA	NA
preserved ~ CumErr	2822.465	0.000000	1.000000	0.6527075	0.0537593	2.348075	- 0.7024110	NA	NA	NA	NA
preserved ~ CumErr	2822.465	3.914534	0.1412439	0.1237631	0.0537593	2.348075	- 0.7024110	NA	NA	NA	NA
preserved ~ CumErr + CumPres	2823.726	6.261923	0.532079	0.3472925	0.0504098	2.403190	- 0.6954848	NA	NA	NA	- 0.0225008
preserved ~ pos	2889.121	70.570588	0.000000	0.000000	0.0228542	2.723778	NA	NA	- 0.1554804	NA	NA
preserved ~ I(pos^2) + pos	2889.587	3.385208	0.000000	0.000000	0.0023570	0.8922003	NA	0.0122337	- 0.2678608	NA	NA
preserved ~ stimlen	2909.769	5.896214	0.000000	0.000000	0.0013183	3.327873	NA	NA	NA	- 0.1583129	NA
preserved ~ CumPres	2933.372	10.907188	0.000000	0.000000	0.0002066	3.221961	NA	NA	NA	NA	- 0.0523184

```
# explore influence of frequency and length

if(grepl("stimlen", BestModelFormulaL2)){ # if length is already in the formula
  Level3ModelEquations<-c(
    BestModelFormulaL2, # best model from level 2
    "preserved ~ 1", # null model
    paste0(BestModelFormulaL2, " + log_freq")
  )
}
```

```

}else if(grepl("log_freq",BestModelFormulaL2)){ # if log_freq is already in the formula
  Level3ModelEquations<-c(
    BestModelFormulaL2, # best model from level 2
    "preserved ~ 1", # null model
    paste0(BestModelFormulaL2," + stimlen")
  )
}else{
  Level3ModelEquations<-c(
    BestModelFormulaL2, # best model from level 2
    "preserved ~ 1", # null model
    paste0(BestModelFormulaL2," + log_freq"),
    paste0(BestModelFormulaL2," + stimlen"),
    paste0(BestModelFormulaL2," + stimlen + log_freq")
  )
}
}

Level3Res<-TestModels(Level3ModelEquations,PosDat)

```

```

## *****
## model index: 3
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      CumErr      stimlen      log_freq
##      3.02570      -0.65298      -0.08838       0.04395
##
## Degrees of Freedom: 4226 Total (i.e. Null); 4223 Residual
## Null Deviance:      2934
## Residual Deviance: 2805 AIC: 2813
## log likelihood: -1402.634
## Nagelkerke R2: 0.05982048
## % pres/err predicted correctly: -797.8444
## % of predictable range [ (model-null)/(1-null) ]: 0.03666981
## *****
## model index: 1
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      CumErr      stimlen
##      3.1238      -0.6568      -0.1013
##
## Degrees of Freedom: 4226 Total (i.e. Null); 4224 Residual
## Null Deviance:      2934
## Residual Deviance: 2808 AIC: 2814
## log likelihood: -1403.934
## Nagelkerke R2: 0.058628
## % pres/err predicted correctly: -798.1602
## % of predictable range [ (model-null)/(1-null) ]: 0.03628898
## *****
## model index: 2

```

```
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)
## 2.088
##
## Degrees of Freedom: 4226 Total (i.e. Null); 4226 Residual
## Null Deviance: 2934
## Residual Deviance: 2934 AIC: 2936
## log likelihood: -1466.873
## Nagelkerke R2: 0
## % pres/err predicted correctly: -828.2529
## % of predictable range [ (model-null)/(1-null) ]: 0
## *****
```

```
BestModelL3<-Level3Res$ModelResult[[ BestModelIndexL3 ]]
BestModelFormulaL3<-Level3Res$Model[[BestModelIndexL3]]

AICSummary<-data.frame(Model=Level3Res$Model,
                        AIC=Level3Res$AIC,
                        row.names = Level3Res$Model)
AICSummary$DeltaAIC<-AICSummary$AIC-AICSummary$AIC[1]
AICSummary$AICexp<-exp(-0.5*AICSummary$DeltaAIC)
AICSummary$AICwt<-AICSummary$AICexp/sum(AICSummary$AICexp)
AICSummary$NagR2<-Level3Res$NagR2

AICSummary <- merge(AICSummary,Level3Res$CoefficientValues,
                    by='row.names',sort=FALSE)
AICSummary <- subset(AICSummary, select = -c(Row.names))

write.csv(AICSummary,paste0(TablesDir,CurPat,"_",RunLabel,"_",
                          CurTask,"_main_effects_plus_one_len_freq_summary.csv"),
          row.names = FALSE)

kable(AICSummary)
```

Model	AIC	DeltaAIC	AICexp	AICwt	NagR2	(Intercept)	CumErr	stimlen	log_freq
preserved ~ CumErr + stimlen + log_freq	2813.2680	0.0000000	1.0000000	0.5743977	0.0598203	3.025704	-	-	0.0439468
preserved ~ CumErr + stimlen	2813.8670	0.5996333	0.7409540	0.4256023	0.0586283	3.123846	-	-	NA
preserved ~ 1	2935.7451	122.4774161	0.0000000	0.0000000	0.0000000	2.088255	NA	NA	NA

```
# BestModelIndex is set by the parameter file and is used to choose
# a model that is essentially equivalent to the lowest AIC model, but
# simpler (and with a slightly higher AIC value, so not in first position)
BestModelL3<-Level3Res$ModelResult[[ BestModelIndexL3 ]]
BestModelFormulaL3<-Level3Res$Model[[BestModelIndexL3]]

BestModel<-BestModelL3
BestModelDeletions<-arrange(dropterm(BestModel),desc(AIC))
```

```

# order by terms that increase AIC the most when they are the one dropped
BestModelDeletions

## Single term deletions
##
## Model:
## preserved ~ CumErr + stimlen + log_freq
##           Df Deviance    AIC
## CumErr    1   2901.9 2907.9
## stimlen    1   2812.8 2818.8
## log_freq   1   2807.9 2813.9
## <none>      1   2805.3 2813.3

#####
# Single deletions from best model
#####

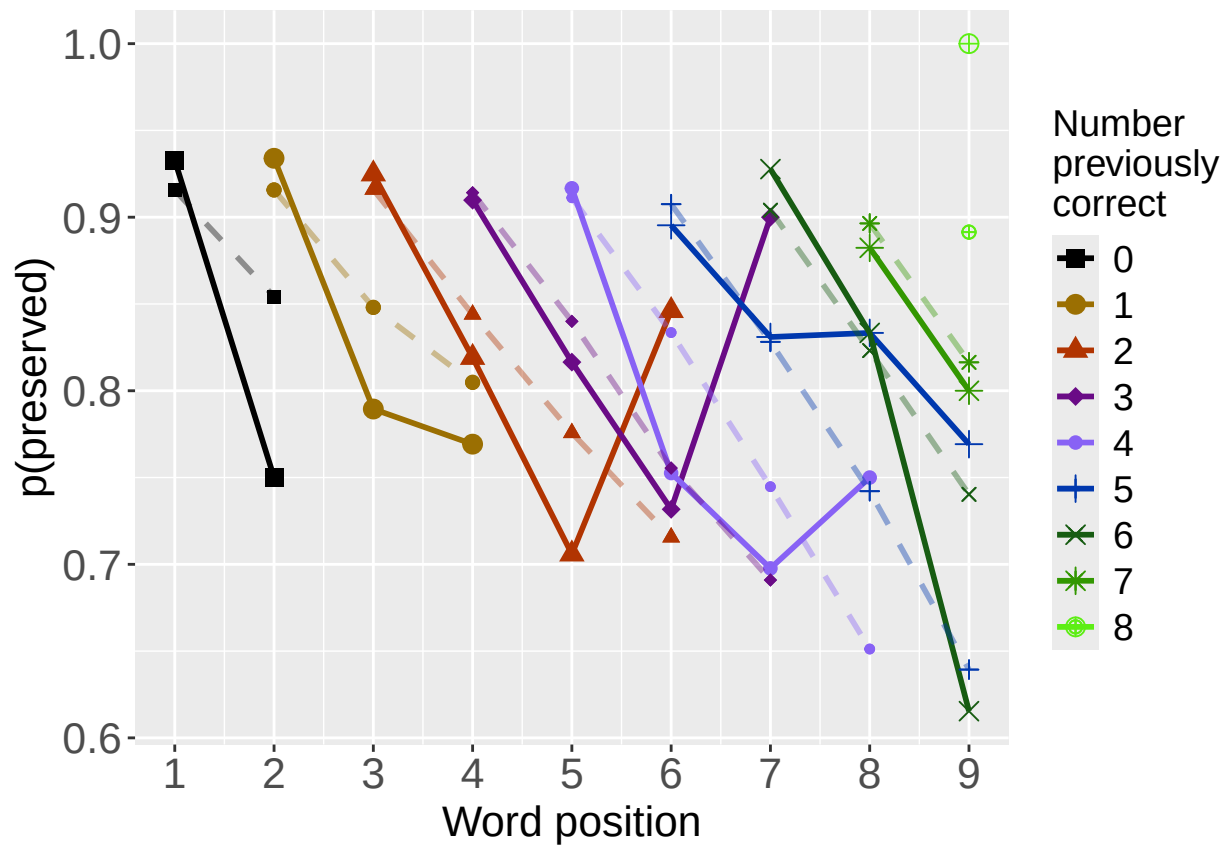
write.csv(BestModelDeletions,paste0(TablesDir,CurPat,"_",
                                   RunLabel,"_",CurTask,
                                   "_best_model_single_term_deletions.csv"),
          row.names = TRUE)

# plot prev err and prev cor with predicted values
PosDat$OAPred<-fitted(BestModel)

PrevCorPlot<-PlotObsPredPreviousCorrect(PosDat,"preserved","OAPred",palette_values,shape_values)

## `summarise()` has grouped output by 'CumPres'. You can override using the `.`groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.`groups` argument.
print(PrevCorPlot)

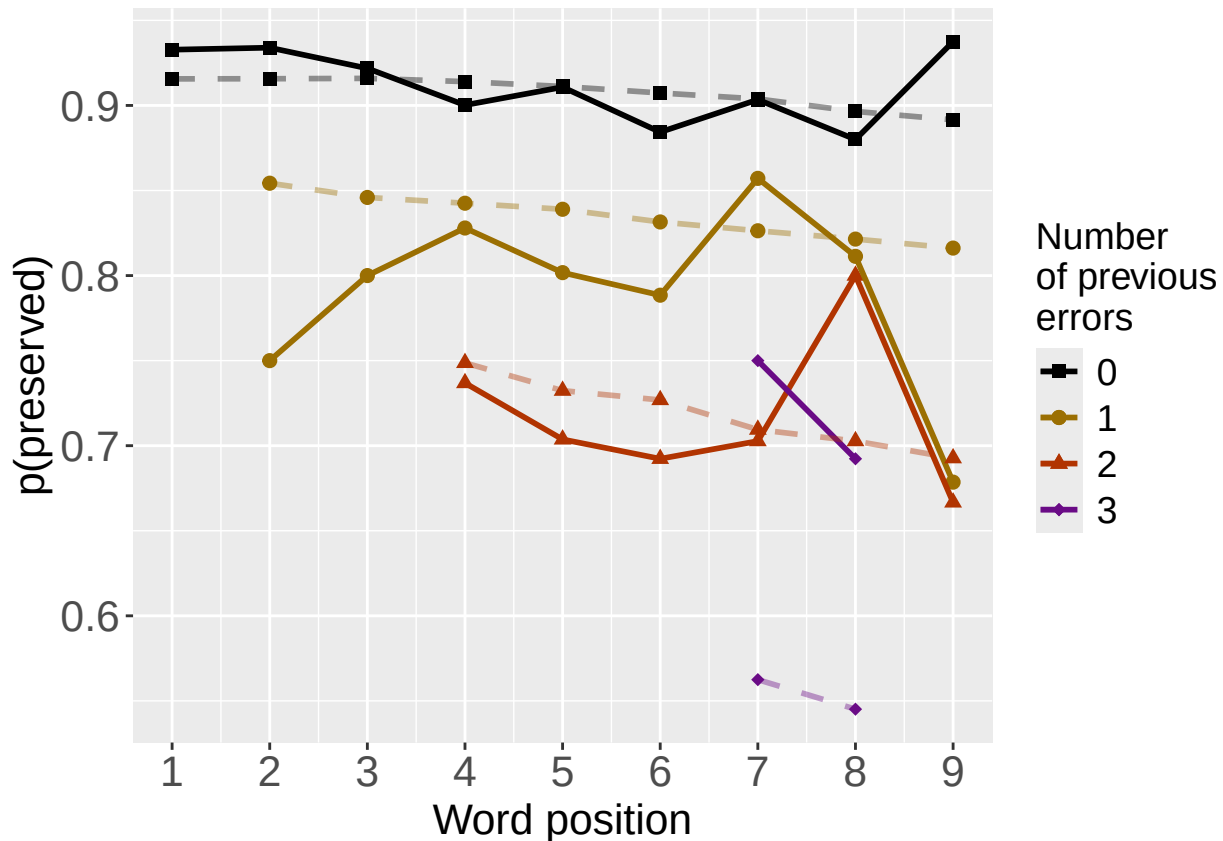
```



```
PrevErrPlot<-PlotObsPredPreviousError(PosDat,"preserved","OAPred",palette_values,shape_values)

## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.

print(PrevErrPlot)
```



```

PlotName<-paste0(FigDir,CurPat,"_",
                  RunLabel,"_",CurTask,
                  "_ObsPred_BestFullModel")
ggsave(filename=paste(PlotName,"_prev_correct.tif",sep=""),plot=PrevCorPlot,device="tiff",compression =

## Saving 6.5 x 4.5 in image
ggsave(filename=paste(PlotName,"_prev_error.tif",sep=""),plot=PrevErrPlot,device="tiff",compression = "

## Saving 6.5 x 4.5 in image
if(DoSimulations){
  BestModelFormulaL3Rnd <- BestModelFormulaL3
  if(grepl("CumPres",BestModelFormulaL3Rnd)){
    BestModelFormulaL3Rnd <- gsub("CumPres","RndCumPres",BestModelFormulaL3Rnd)
  }
  if(grepl("CumErr",BestModelFormulaL3Rnd)){
    BestModelFormulaL3Rnd <- gsub("CumErr","RndCumErr",BestModelFormulaL3Rnd)
  }

  RndModelAIC<-numeric(length=RandomSamples)
  for(rindex in seq(1,RandomSamples)){
    # Shuffle cumulative values
    PosDat$RndCumPres <- ShuffleWithinWord(PosDat,"CumPres")
    PosDat$RndCumErr <- ShuffleWithinWord(PosDat,"CumErr")
    BestModelRnd <- glm(as.formula(BestModelFormulaL3Rnd),
                       family="binomial",data=PosDat)
    RndModelAIC[rindex] <- BestModelRnd$aic
  }
}

```

```

}
ModelNames<-c(paste0("***",BestModelFormulaL3),
              rep(BestModelFormulaL3Rnd,RandomSamples))
AICValues <- c(BestModelL3$aic,RndModelAIC)
BestModelRndDF <- data.frame(Name=ModelNames,AIC=AICValues)
BestModelRndDF <- BestModelRndDF %>% arrange(AIC)
BestModelRndDF <- rbind(BestModelRndDF,
                        data.frame(Name=c("Random average"),
                                   AIC=c(mean(RndModelAIC))))
BestModelRndDF <- rbind(BestModelRndDF,
                        data.frame(Name=c("Random SD"),
                                   AIC=c(sd(RndModelAIC))))
write.csv(BestModelRndDF,paste0(TablesDir,CurPat,"_",RunLabel,"_",CurTask,
                                "_best_model_with_random_cum_term.csv"),
          row.names = FALSE)
}

# plot influence factors
num_models <- nrow(BestModelDeletions) - 1
# minus 1 because last
# model is always <none> and we don't want to include that

if(num_models > 4){
  last_model_num <- 4
}else{
  last_model_num <- num_models
}

FinalModelSet<-ModelSetFromDeletions(BestModelFormulaL3,
                                     BestModelDeletions,
                                     last_model_num)

PlotName<-paste0(FigDir,CurPat,"_",RunLabel,"_",CurTask,"_FactorPlots")

FactorPlot<-PlotModelSetWithFreq(PosDat,FinalModelSet,
                                 palette_values,FinalModelSet,PptID=CurPat)

## *****
## model index: 1
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##          data = PosDat)
##
## Coefficients:
## (Intercept)      CumErr
##      2.3481      -0.7024
##
## Degrees of Freedom: 4226 Total (i.e. Null); 4225 Residual
## Null Deviance:      2934
## Residual Deviance: 2818 AIC: 2822
## log likelihood: -1409.232
## Nagelkerke R2: 0.05375928
## % pres/err predicted correctly: -799.5156
## % of predictable range [ (model-null)/(1-null) ]: 0.03465446

```



```

## *****
## model index: 2
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      CumErr      stimlen
##      3.1238      -0.6568      -0.1013
##
## Degrees of Freedom: 4226 Total (i.e. Null); 4224 Residual
## Null Deviance:      2934
## Residual Deviance: 2808 AIC: 2814
## log likelihood: -1403.934
## Nagelkerke R2: 0.058628
## % pres/err predicted correctly: -798.1602
## % of predictable range [ (model-null)/(1-null) ]: 0.03628898
## *****
## model index: 3
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      CumErr      stimlen      log_freq
##      3.02570      -0.65298      -0.08838      0.04395
##
## Degrees of Freedom: 4226 Total (i.e. Null); 4223 Residual
## Null Deviance:      2934
## Residual Deviance: 2805 AIC: 2813
## log likelihood: -1402.634
## Nagelkerke R2: 0.05982048
## % pres/err predicted correctly: -797.8444
## % of predictable range [ (model-null)/(1-null) ]: 0.03666981
## *****

## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'freq_bin'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'freq_bin'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'freq_bin'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'freq_bin'. You can override using the `.groups` argument.

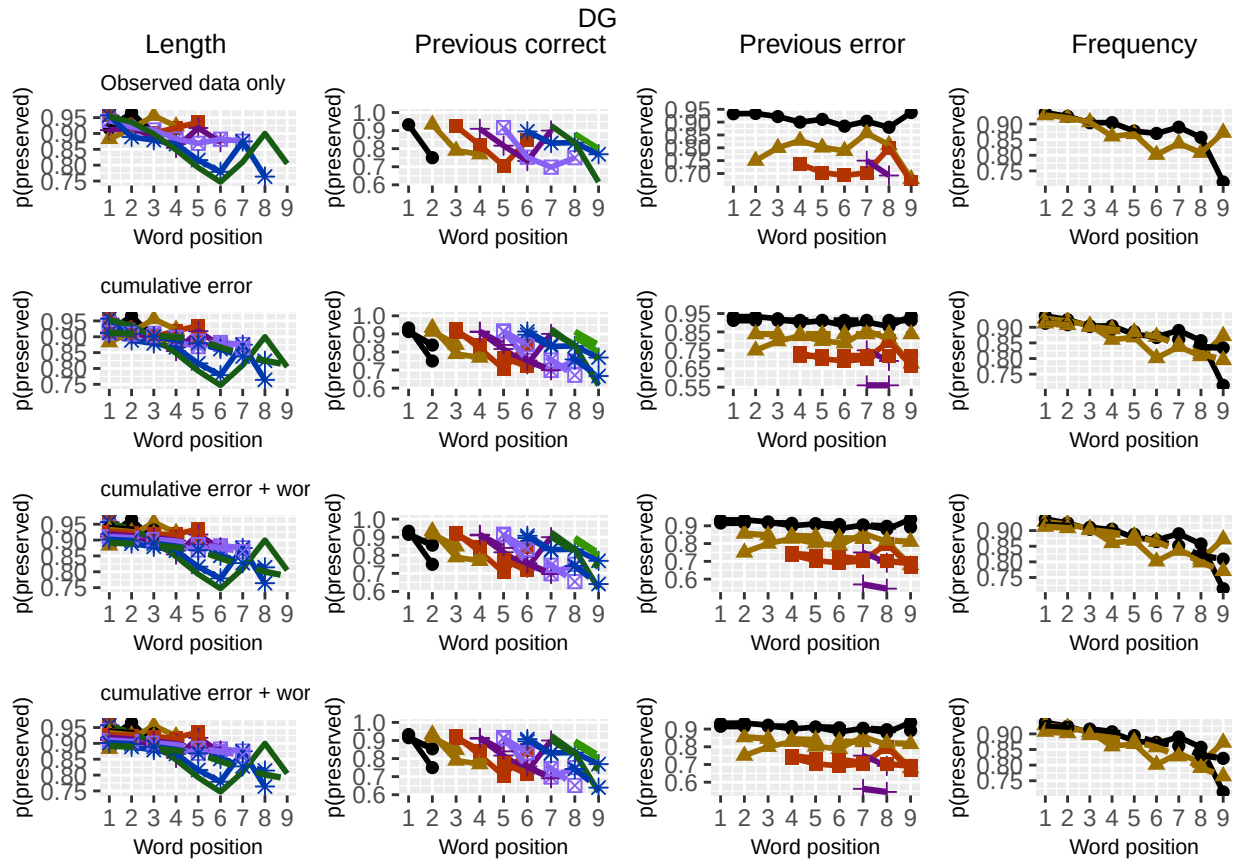
## Warning: The shape palette can deal with a maximum of 6 discrete values because more than 6 becomes
## i you have requested 7 values. Consider specifying shapes manually if you need that many have them.

```

```

## Warning: Removed 9 rows containing missing values or values outside the scale range (`geom_point()`).
## Warning: The shape palette can deal with a maximum of 6 discrete values because more than 6 becomes
## i you have requested 9 values. Consider specifying shapes manually if you need that many have them.
## Warning: Removed 6 rows containing missing values or values outside the scale range (`geom_point()`).
## Warning: The shape palette can deal with a maximum of 6 discrete values because more than 6 becomes
## i you have requested 7 values. Consider specifying shapes manually if you need that many have them.
## Warning: Removed 9 rows containing missing values or values outside the scale range (`geom_point()`).
## Removed 9 rows containing missing values or values outside the scale range (`geom_point()`).
## Warning: The shape palette can deal with a maximum of 6 discrete values because more than 6 becomes
## i you have requested 9 values. Consider specifying shapes manually if you need that many have them.
## Warning: Removed 6 rows containing missing values or values outside the scale range (`geom_point()`).
## Removed 6 rows containing missing values or values outside the scale range (`geom_point()`).
## Warning: The shape palette can deal with a maximum of 6 discrete values because more than 6 becomes
## i you have requested 7 values. Consider specifying shapes manually if you need that many have them.
## Warning: Removed 9 rows containing missing values or values outside the scale range (`geom_point()`).
## Removed 9 rows containing missing values or values outside the scale range (`geom_point()`).
## Warning: The shape palette can deal with a maximum of 6 discrete values because more than 6 becomes
## i you have requested 9 values. Consider specifying shapes manually if you need that many have them.
## Warning: Removed 6 rows containing missing values or values outside the scale range (`geom_point()`).
## Removed 6 rows containing missing values or values outside the scale range (`geom_point()`).
## Warning: The shape palette can deal with a maximum of 6 discrete values because more than 6 becomes
## i you have requested 7 values. Consider specifying shapes manually if you need that many have them.
## Warning: Removed 9 rows containing missing values or values outside the scale range (`geom_point()`).
## Removed 9 rows containing missing values or values outside the scale range (`geom_point()`).
## Warning: The shape palette can deal with a maximum of 6 discrete values because more than 6 becomes
## i you have requested 9 values. Consider specifying shapes manually if you need that many have them.
## Warning: Removed 6 rows containing missing values or values outside the scale range (`geom_point()`).
## Removed 6 rows containing missing values or values outside the scale range (`geom_point()`).
ggsave(paste0(PlotName, ".tif"), plot=FactorPlot, width = 360, height=400, units="mm", device="tiff", compress=
# use \blandscape and \elandscape to make markdown plots landscape if needed
FactorPlot

```



```
DA.Result<-dominanceAnalysis(BestModel)
DAContributionAverage<-ConvertDominanceResultToTable(DA.Result)
write.csv(DAContributionAverage,paste0(TablesDir,CurPat,"_",RunLabel,"_",
                                     CurTask,"_dominance_analysis_table.csv"),
          row.names = TRUE)
kable(DAContributionAverage)
```

	CumErr	stimlen	log_freq
McFadden	0.0358854	0.0058118	0.0020958
SquaredCorrelation	0.0245038	0.0039933	0.0014401
Nagelkerke	0.0245038	0.0039933	0.0014401
Estrella	0.0250888	0.0040520	0.0014612

	deviance	deviance_explained
CumErr + stimlen + log_freq	2805.268	128.4774
CumErr + stimlen	2807.867	125.8778
CumErr	2818.465	115.2805
null	2933.745	0.0000

	percent_explained	percent_of_explained_deviance	increment_in_explained
CumErr + stimlen + log_freq	4.379297	100.00000	2.023417
CumErr + stimlen	4.290686	97.97658	8.248400
CumErr	3.929464	89.72818	89.728184
null	0.000000	NA	0.000000

```
deviance_differences_df<-GetDevianceSet(FinalModelSet,PosDat)
write.csv(deviance_differences_df,paste0(TablesDir,CurPat,"_",RunLabel,"_",
                                         CurTask,"_deviance_differences_table.csv"),
          row.names = FALSE)
deviance_differences_df
```

```
##                                model deviance deviance_explained percent_explained percent_of_explained_deviance
## CumErr + stimlen + log_freq CumErr + stimlen + log_freq 2805.268          128.4774          4.379297          100.00000
## CumErr + stimlen          CumErr + stimlen 2807.867          125.8778          4.290686          97.97658
## CumErr          CumErr 2818.465          115.2805          3.929464          89.72818
## null          null 2933.745          0.0000          0.000000          NA
##                                increment_in_explained
## CumErr + stimlen + log_freq          2.023417
## CumErr + stimlen          8.248400
## CumErr          89.728184
## null          0.000000
```

```
kable(deviance_differences_df[,2:3], format="latex", booktabs=TRUE) %>%
  kable_styling(latex_options="scale_down")
```

```
kable(deviance_differences_df[,c(4:6)], format="latex", booktabs=TRUE) %>%
  kable_styling(latex_options="scale_down")
```

```
NagContributions <- t(DAContributionAverage[3,])
NagTotal<-sum(NagContributions)
```

```

NagPercents <- NagContributions / NagTotal
NagResultTable <- data.frame(NagContributions = NagContributions, NagPercents = NagPercents)
names(NagResultTable)<-c("NagContributions","NagPercents")
write.csv(NagResultTable,paste0(TablesDir,CurPat,"_",RunLabel,"_",CurTask,
                                "_nagelkerke_contributions_table.csv"),row.names = TRUE)
NagPercents

```

```

##           Nagelkerke
## CumErr    0.81850714
## stimlen   0.13338844
## log_freq  0.04810442

```

```

sse_results_list<-compare_SS_accounted_for(FinalModelSet,"preserved ~ 1",PosDat,N_cutoff=10)

```

```

## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.

## Warning in (cumerr_residuals^2) * (N_values$cumerr_N): longer object length is not a multiple of shorter object length
## Warning in (null_cumerr_resid^2) * (N_values$len_pos_N): longer object length is not a multiple of shorter object length
## Warning in (null_cumcor_resid^2) * (N_values$len_pos_N): longer object length is not a multiple of shorter object length

## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.

## Warning in (cumerr_residuals^2) * (N_values$cumerr_N): longer object length is not a multiple of shorter object length

```

model	p_accounted_for	model_deviance
preserved ~ CumErr+stimlen	0.8140032	2807.867
preserved ~ CumErr+stimlen+log_freq	0.8151657	2805.268
preserved ~ CumErr	0.8234225	2818.465

```
## Warning in (null_cumerr_resid^2) * (N_values$len_pos_N): longer object length is not a multiple of shorter object length
## Warning in (null_cumcor_resid^2) * (N_values$len_pos_N): longer object length is not a multiple of shorter object length
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
## Warning in (cumerr_residuals^2) * (N_values$cumerr_N): longer object length is not a multiple of shorter object length
## Warning in (null_cumerr_resid^2) * (N_values$len_pos_N): longer object length is not a multiple of shorter object length
## Warning in (null_cumcor_resid^2) * (N_values$len_pos_N): longer object length is not a multiple of shorter object length
```

```
sse_table <- sse_results_table(sse_results_list)
write.csv(sse_table, paste0(TablesDir, CurPat, "_", RunLabel, "_",
                           CurTask, "_sse_results_table.csv"), row.names = TRUE)
sse_table
```

```
##              model p_accounted_for model_deviance diff_CumErr+stimlen diff_CumErr+stimlen+log_freq diff_CumErr
## 1      preserved ~ CumErr+stimlen      0.8140032      2807.867           0.000000000          -0.001162451 -0.009419327
## 2 preserved ~ CumErr+stimlen+log_freq      0.8151657      2805.268           0.001162451           0.000000000 -0.008256876
## 3           preserved ~ CumErr      0.8234225      2818.465           0.009419327           0.008256876  0.000000000
```

```
kable(sse_table[,1:3], format="latex", booktabs=TRUE) %>%
  kable_styling(latex_options="scale_down")
```

```
write.csv(results_report_DF, paste0(TablesDir, CurPat, "_", RunLabel, "_",
                                   CurTask, "_results_report_df.csv"), row.names = FALSE)
```

```
ncols_in_table <- ncol(sse_table)
kable(sse_table[,c(1,4:ncols_in_table)], format="latex", booktabs=TRUE) %>%
  kable_styling(latex_options="scale_down")
```

model	diff_CumErr+stimlen	diff_CumErr+stimlen+log_freq	diff_CumErr
preserved ~ CumErr+stimlen	0.0000000	-0.0011625	-0.0094193
preserved ~ CumErr+stimlen+log_freq	0.0011625	0.0000000	-0.0082569
preserved ~ CumErr	0.0094193	0.0082569	0.0000000